



accenture

dfki
ai Deutsches Forschungszentrum
für Künstliche Intelligenz
German Research Center for
Artificial Intelligence

Alassane Ndiaye, Xhorxhina Taraj, Bostjan
Robeznik, Gerd Herzog, Christian Gerber,
Hendrik Purwins

Generative AI in Software Engineering:

Transforming the Software Development Process

Oktober 2025



Executive Summary

In this paper, we assess the current and expected impact of generative AI (GenAI) on software engineering (SE), in particular on initial requirements gathering and design, implementation (including code generation, suggestion, and explanation), as well as from integration and testing to continuous deployment and support (code review, repair, and maintenance). The paper contains a review of an extensive amount of current scientific research on this topic. We exemplify the use of large language models (LLMs) in SE with a practical example and include a survey on the productive and efficient use of GenAI. The collaboration between human software engineer and AI tools is at the core of effective integration of GenAI into SE. AI can take over formalized, technical, repetitive, error-prone, and effort-intensive tasks. Human developers should focus on well-defined problem formulations, a creative outlining of the solution structure, its orchestration and supervision as well as a critical supervision and validation of AI-generated suggestions. In a holistic awareness of the different phases of SE, LLMs can fully unlock the impact of GenAI on productivity, efficiency, and innovation. Future challenges comprise questions of intellectual property and privacy of code and data, validation of AI suggestions (transparency and explainability of AI suggestions, availability of data sets), and the constant adaptation to the rapidly evolving field of AI.



Table of Contents

1.	Introduction	5
2.	Software Engineering and Generative AI Foundations	9
2.1	Software Engineering Research and Practice	10
2.2	General Tools and Methods for Software Engineering.....	11
2.3	Aspects of Software Engineering	11
2.4	Generative AI based on Large Language Models.....	12
2.4.1	Large Language Models.....	12
2.4.2	Datasets for Code-Centric LLM Creation and Evaluation	15
2.4.3	LLMs Adjusted for Software Engineering.....	16
2.5	Metrics for Evaluation	17
3.	Assessment of Core Tasks in Software Engineering	19
3.1	Infrastructure and Tools for AI-based Development	20
3.1.1	The Landscape of AI Tooling for Software Development	20
3.1.2	Generative AI Infrastructure Considerations.....	24
3.1.3	Evolution of Development Tools and Future Directions.....	25
3.2	Analysis and Design	26
3.2.1	Requirement Engineering.....	26
3.2.2	UI/UX Design	28
3.2.3	Software Design.....	29
3.3	Implementation	30
3.3.1	Code Generation	30
3.3.2	Code Suggestion	31
3.3.3	Code Explanation.....	32
3.4	Integration and Testing.....	33
3.4.1	Software Integration.....	33
3.4.2	Testing.....	33
3.5	Operations and Maintenance	36
3.5.1	Code Review	36
3.5.2	Program Repair.....	36
3.5.3	Code Maintenance	37
3.6	AI-based Software Development Tools for Specific Engineering Tasks: Accenture Practice	39
3.6.1	Study on Adoption of GitHub Copilot.....	39



3.6.2	Study on AI Use for Different Software Engineering Tasks	39
3.6.3	Accenture GenWizard Tool	40
4.	Integration of Generative AI Across the Software Development Process	41
4.1	Large Language Models as General-Purpose Tools in Software Engineering.....	42
4.2	Human-AI Interaction in Software Development	44
4.2.1	Impact of AI Coding Assistants on Productivity and Efficiency in Development Workflows.....	44
4.2.2	Collaboration Patterns Between Human Developers and AI Systems	44
4.2.3	Organizational and Process-Level Adaptations for AI Integration.....	45
4.2.4	Human Oversight, Prompting Strategies, and Feedback Loops with AI	45
4.2.5	Productivity, Efficiency, Creativity, and Challenges in AI-Assisted Development	46
4.2.6	Key Areas of Impact	47
4.3	Case Study: Continuous LLM Engagement for Warehouse Routing Optimization	48
4.3.1	Example User Story	48
4.3.2	Experiment Environment.....	49
4.3.3	Phase 1: Requirements Analysis	49
4.3.4	Phase 2: Technology Evaluation & Refinement.....	50
4.3.5	Phase 3: Rapid Prototyping.....	50
4.3.6	Phase 4: Validation, Testing & Optimization	52
4.3.7	Phase 5: Documentation & Knowledge Capture	52
4.3.8	Phase 6: Deployment & Continuous Improvement	53
4.3.9	Current Landscape & Future Potential	53
4.3.10	Practical Mitigation of Context Window Limits.....	54
4.3.11	Key Take Away.....	55
4.4	Adoption of Generative AI into Software Development.....	56
4.4.1	Challenges and Considerations	56
4.4.2	Recommendations for Implementation	57
4.4.3	Future Considerations	58
5.	Conclusion	59
5.1	Toward AI-Enhanced Software Development	60
5.2	Outlook	61
6.	Acknowledgements	62
7.	References.....	63
8.	Authors.....	79

1. Introduction



The illustration shows a man with a beard and glasses, wearing a teal shirt, sitting at a desk and looking at a computer monitor. A woman with dark hair, wearing an orange shirt, is standing next to him, pointing at the screen with her right index finger. The monitor displays a block of code in a light blue font on a dark blue background. The background of the entire image is a solid purple color with faint, light blue geometric patterns (hexagons and lines) scattered across it.

```
int +  
function () {  
  function () {  
    void function () run ()  
    if ()  
  } return i;  
} else {  
  return r;  
}
```



The current transformation in software development is succinctly described by catchy social media posts such as “English is the new most important programming language” (Karpathy 2023) and the introduction of new terms like “vibe coding” (Karpathy 2025a). In his keynote speech to aspiring AI entrepreneurs, visionary AI researcher Andrej Karpathy introduced “Software 3.0” (Karpathy 2025b): his concept for a new era of software engineering that uses generative artificial intelligence (GenAI). In this emerging era, natural language will become the new programming interface, and AI models will handle the rest. Opinions on the impact of new technologies based on large language models (LLMs) range from “The end of programming” (Welsh 2023) to “Why it is unlikely new developments in machine intelligence will eventually make programming obsolete” (Hüttel 2025).

Hurlburt (2025) examined the key scientific developments and the parallel technological advancements that established the “Information Age”. This scientific progress enabled generative AI and Hurlburt characterizes ChatGPT as a significant technological advancement in this evolution. The “Malicious AI Report” (Brundage et al. 2018) was written by authors from various institutions spanning academia, civil society, and industry, including OpenAI. The report examined potential security threats resulting from the malicious use of AI and suggested methods for forecasting, preventing, and mitigating these threats. The European Commission’s Joint Research Centre (JRC) has prepared an “Outlook” report that provides a comprehensive overview of the technological aspects of GenAI (Abendroth-Dias et al. 2025). This report includes detailed information on their current capabilities and emerging trends.

METR (Model Evaluation and Threat Research) studies, develops, and conducts evaluations of the ability of cutting-edge AI systems to perform complex tasks without human input. According to the “task-completion time horizon” metric devised by METR (Kwa et al. 2025), the capabilities of key LLMs double every seven months. Given this exponential improvement, the most advanced LLMs in 2030 should be able to reliably complete a software-based task that would take humans a full month of 40-hour workweeks (Zorpette 2025). These LLMs will likely complete these tasks much more quickly than humans, in just days or hours. However, there are limits to this kind of predictive approach (Lovely 2025). For instance, the metric might not reveal how well models can “generalize” by performing tasks different from those they were trained on. Kokotajlo et al. (2025) from the AI Futures project developed a scenario forecasting the realization of artificial general intelligence (AGI), which predicts coding automation by early 2026.

Our joint study, conducted within the Accenture AI Partner Lab at DFKI, aims to provide balanced insights into the opportunities and challenges of using generative AI in software engineering. GenAI represents a paradigm shift with the potential to profoundly impact software development practices. This white paper presents a comprehensive assessment of the impact of GenAI on all phases of the software development lifecycle, from initial requirements gathering to continuous delivery, operations and maintenance. The white paper provides a comprehensive overview of the current scientific literature on the application of generative artificial intelligence in the field of software development. It includes a practical demonstration using a real-world use case study and a discussion of the potential benefits of human-AI collaboration in software development, as well as the impact on the four key dimensions that we consider in our study.



Figure 1.1 shows the conceptual overview of our approach. We focus on the following key dimensions of generative AI use in software development: productivity, quality, creativity and efficiency. Productivity and efficiency are based on the effort and resources used. These two dimensions are interdependent and complementary. Productivity increases when more results are achieved with the same resources (e.g., time spent on a particular activity). Higher efficiency means achieving the same results with fewer resources. In other words, you need a smaller budget, fewer staff, or less time to achieve the same output. However, higher productivity and efficiency does not necessarily mean that the quality of the end result is good enough.

Effectiveness does not refer to the quantity of results achieved, or the quantity of resources used. Rather, it is linked to quality; it is about doing the right thing—choosing tasks wisely so they produce the best results. Creativity, on the other hand, is a key dimension of software development that enables innovative solutions.

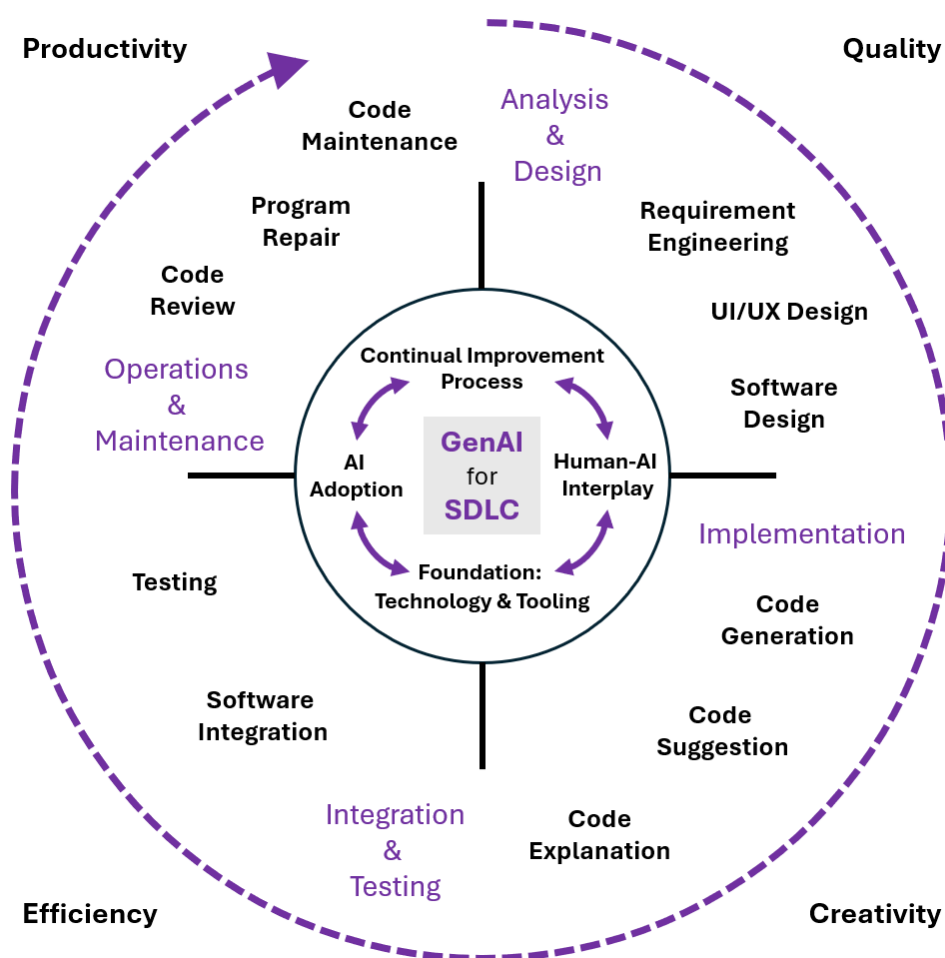


Figure 1.1: Generative AI (GenAI) in the software development life cycle (SDLC): Selected task groups and building blocks for realization.



The subsequent Chapter 2 establishes the context for understanding the core principles of software engineering. It describes the GenAI technology foundation, as well as the metrics for evaluation. Starting with an overview of the landscape of AI tooling for software development as the second building block of our GenAI foundation, Chapter 3 provides a comprehensive evaluation of the impact of generative AI on core software development activities. Our specific segmentation of the software development life cycle into four main task groups (analysis & design, implementation, integration & testing and operations & maintenance) is illustrated in Figure 1.1. Chapter 4 examines the integration of generative AI across the entire software development process (see the inner circle in Figure 1.1). The discussion will cover the interplay between human developers and AI components—designed to ensure collaboration and human oversight, the successful adoption of generative AI in the enterprise, and the necessary process of continual improvement, all of which are based on the established GenAI technology and tooling foundation. Finally, Chapter 5 concludes the paper by discussing the potential of AI-enhanced software development and outlining future research challenges.

The landscape of AI-based technologies changes rapidly, almost on a weekly basis, with new models, techniques, and approaches constantly appearing and disappearing. This white paper concludes our joint study and provides background knowledge and current information as of late summer 2025.



2. Software Engineering and Generative AI Foundations





Generative artificial intelligence (GenAI), particularly driven by large language models (LLMs), is rapidly emerging as a transformative technology, with significant potential to revolutionize various domains.

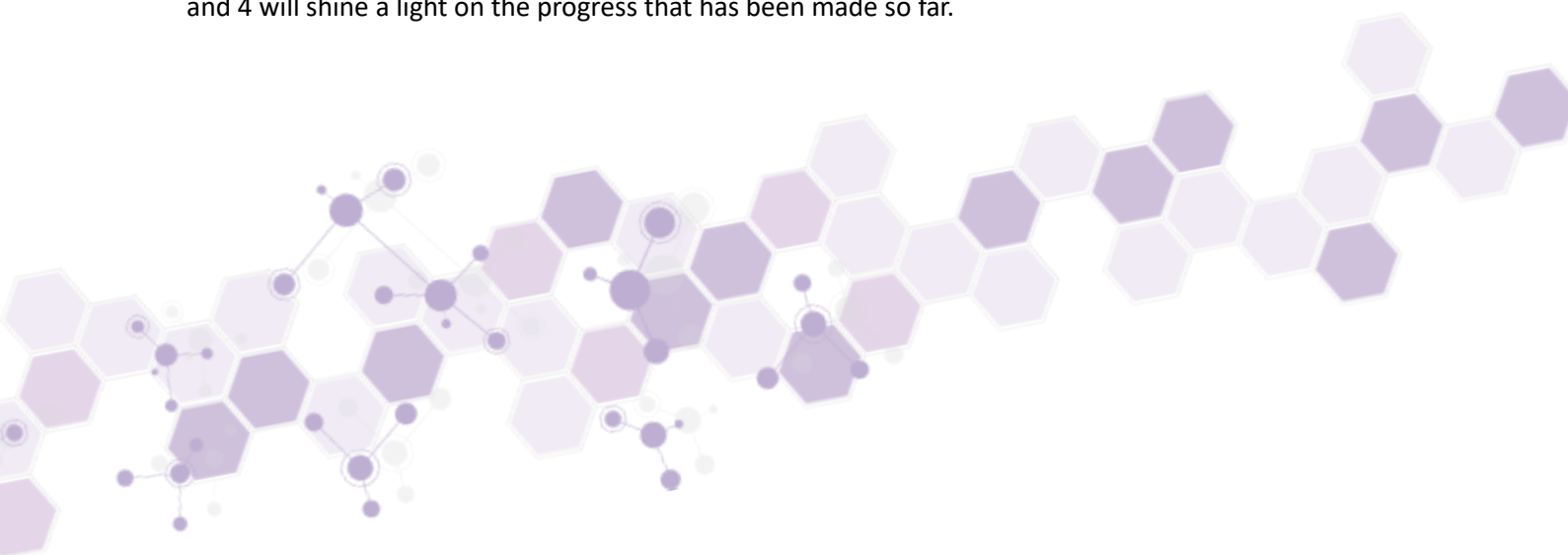
This paper explores the opportunities and challenges presented by GenAI specifically within the field of software engineering. By first establishing the fundamental principles and practices of software engineering, we set the stage for a detailed evaluation of how GenAI technologies can enhance core software development tasks and processes.

2.1 Software Engineering Research and Practice

Software engineering is characterized by systematic, measurable, and repeatable methods for developing high-quality software within predefined time and budget constraints. This discipline, as comprehensively outlined in the Guide to the Software Engineering Body of Knowledge (SWEBOK)—edited by Washizaki (2024)—has evolved significantly since its inception in the 1960s. Initially emerging as a structured approach to address the complexities inherent in software development, software engineering integrates technical, managerial, and interdisciplinary practices to effectively manage and streamline the creation and maintenance of software products.

The continual evolution of software engineering practices provides a foundational context to assess how emerging technologies, notably GenAI, can be integrated into established processes to enhance productivity, efficiency, reliability, and overall software quality.

The interplay between innovations in artificial intelligence (AI) and software technology has had a positive impact on progress in both areas. The Software Engineering Institute (SEI) at Carnegie Mellon University has produced a study on the future of software engineering (Carleton, Klein, et al. 2021) that provides a 10–15 year research roadmap. According to the SEI study, the emerging vision is that the current notion of software development will be replaced. The software pipeline of the future realizes the idea that “humans and AI are trustworthy collaborators that rapidly evolve systems based on programmer intent” (Carleton, Shull, and Harper 2022). The rise of generative AI will help to pave the way for this long-term vision and to fundamentally change the way software is produced. Chapters 3 and 4 will shine a light on the progress that has been made so far.





2.2 General Tools and Methods for Software Engineering

Advancements in software engineering have been driven by new programming paradigms, methodologies, and software tools designed to improve efficiency, adaptability, and productivity. A significant shift occurred with the publication of the Agile Manifesto (Beck et al. 2001), introducing agile software development practices characterized by iterative processes, rapid feedback, and responsiveness to change. Similarly, the emergence of DevOps methodologies, notably advocated by Allspaw and Hammond (2009), emphasized the seamless integration of development and operational processes to facilitate faster and more reliable software delivery cycles. The strong emphasis on agile and DevOps is justified by their alignment with the iterative nature and generative capabilities of GenAI, enabling teams to rapidly incorporate AI-driven innovations into software workflows.

Modern software engineering relies heavily on automation to enhance the reliability and efficiency of development processes. Integrated Development Environments (IDEs), automated testing frameworks, revision control systems, and issue tracking tools have become essential for maintaining consistency, quality control, efficiency and productivity. The distributed version control system Git, initially released in 2005, has become a standard tool for source code management (Chacon and Straub 2014). Cloud-based platforms like GitHub and GitLab further extend these capabilities, providing comprehensive solutions for scalable collaboration, automated build pipelines, and continuous integration and continuous delivery (CI/CD) practices (Duvall, Matyas, and Glover 2007; Fowler 2024).

These tools and methodologies form a cohesive framework, ideally suited to leverage the capabilities of GenAI. This synergy facilitates innovation in software development by automating routine tasks and enabling developers to focus more on creative and strategic aspects of software engineering.

2.3 Aspects of Software Engineering

Existing software engineering methodologies and paradigms differ in their specific structuring into core activities, processes, and life cycle phases. According to Washizaki (2024), the software engineering process encompasses all the activities involved in conceiving, designing, implementing, testing, deploying, and maintaining software systems. The scope covers the entire software life cycle, from inception to retirement.

Our own focus combines aspects from agile and lean with continuous delivery. For our assessment of the use of generative AI for software engineering tasks, we apply a pragmatic decomposition into four main categories of major process steps (see also Figure 1.1). We will consider core development activities in Analysis and Design, Implementation, Integration and Testing as well as in Operations and Maintenance. In addition, we also take a deeper look at the potential impact across the entire software life cycle. Life cycle improvements should be driven by analyzing process and product metrics. Continuous assessment and improvement of software engineering processes is essential to enable organizations to develop software systems more efficiently and at lower cost.



2.4 Generative AI based on Large Language Models

While earlier machine learning methods often focus on tasks such as classification, regression, dimension reduction, or clustering in a specific context, generative AI is characterized by its ability to generate content in response to a prompt, a textual query that can be written in natural language. The generated content can be images, video, audio, and text, including computer code or design concepts. This approach has opened up a large number of new applications in a wide range of fields, including software engineering.

2.4.1 Large Language Models

Text-based generative AI is primarily implemented as large language models (LLMs). LLMs are models that represent words and texts as vectors, meaning that semantically similar content is represented by neighboring vectors in a high-dimensional space. LLMs are neural networks consisting of numerous layers. Each layer is made up of values computed from the previous layers using parameters (or weights). Recurrent connections between neurons in the encoding as well as the decoding layer and the attention mechanism (Vaswani et al. 2017) model dependencies between different parts of a text and form components of the Transformer architecture. Prominent LLMs include the model families GPT (OpenAI), LLaMA (Meta), Qwen (Alibaba), Claude (Anthropic), DeepSeek and Mistral.

Training and Alignment

The training of LLMs works as follows: First, a corpus of text data is required. The training takes place step by step. In a single training step, an existing text is used, with a part of this text being omitted. The LLM then generates a text segment to replace the omitted part. The model then compares the generated segment with the omitted one. The model's parameters are then adjusted so that the generated and omitted segments are as semantically similar as possible. The entire training process consists of a very large number of such training steps, in which different parts from a large number of texts are omitted and replaced, and the parameters are further optimized accordingly.

Common LLMs have between 7 billion and 1.8 trillion parameters (GPT-4: estimated at 1.76 trillion, GPT-4o: 175 billion). For the model LLaMA-2, the training data consists of approximately 4.5 terabytes of data; for GPT-4, it is more than 1 petabyte. The training of GPT-4 costs around 100 million dollars (Meyer 2024).

As generative AI models become larger and more powerful, some scientists are calling for more energy-efficient, leaner systems (Ananthaswamy 2023). Shi, Z. Yang, and Lo (2025) provided a comprehensive survey on efficient and green LLMs for software engineering.

An important aspect of developing LLMs that interact with humans is to align their behavior so that it is both useful and harmless to users. In a typical multi-stage LLM training process, this is achieved by adjusting the model to reinforce desirable behaviors and discourage undesirable ones. This facet of the training process is called alignment. Wolf et al. (2024) formally investigated the inherent characteristics and limitations of alignment in LLMs.



Capabilities and Limitations

Ferrando et al. (2024) provided a technical introduction to the techniques currently used to interpret Transformer-based language models, offering an overview of their known internal mechanisms. Essentially, LLMs are stochastic string-generation tools that excel at predicting the next token. Bender et al. (2021) discussed the potential risks of LLM technology, encouraging research beyond ever-larger language models. Despite their powerful capabilities, generative pre-trained transformers (GPTs) tend to “hallucinate” responses (Waldo and Boussard 2025). Hallucinations occur when an LLM-based GPT generates a seemingly realistic yet non-factual, nonsensical, or inconsistent response to a given prompt. The new GPT-5 model (OpenAI 2025) is an important improvement that is reported to significantly reduce hallucinations (Romero 2025).

However, encouraging truthfulness through supervision or reinforcement has only been partially successful. A general method for detecting hallucinations in LLMs is needed that can work with new and unseen questions to which humans might not know the answer. Farquhar et al. (2024) proposed measuring uncertainty about the meanings of generated responses rather than the text itself to improve question-answering accuracy. They developed methods providing uncertainty estimators for LLMs to detect a subset of hallucinations, called confabulations, which are arbitrary and incorrect generations.

LUNA is a universal, model-based analysis framework designed to improve the interpretability, reliability, and trustworthiness of LLMs (Song et al. 2024; Mastropaolo 2025). This extensible framework allows for systematic analysis from multiple quality perspectives in a human-interpretable manner. It addresses key challenges, such as hallucinations, biases, and inconsistencies.

Prompt engineering involves creating specific and effective prompts that guide generative AI models, such as LLMs, to produce the desired output. It means designing the input to an AI system in a way that elicits the most appropriate response, whether the output is text, images, or other types of content. Sasaki et al. (2025) conducted a systematic review of recent literature to create a practical taxonomy of prompt engineering patterns in software engineering.

Jason Wei et al. (2022) demonstrated that generating a series of intermediate reasoning steps, or a “chain of thought” (CoT), significantly improves LLMs’ ability to perform complex reasoning. This led to a simple method called chain-of-thought prompting, in which a few CoT demonstrations are provided as exemplars in prompting.

Lewis et al. (2020) proposed retrieval augmented generation (RAG) to combine information retrieval and generative models. Injecting relevant, contextual supporting information into the prompt enables the LLM to generate meaningful, contextually accurate responses to user input. Since the original proposal, RAG techniques have been constantly refined and improved. Yunfan Gao et al. (2024) and Fan et al. (2024) provide comprehensive surveys.





From LLM-Based Agents to Agentic AI

Proven AI techniques such as rule-based systems and natural language processing have long played an important role in automating business processes across all industries. Numerous providers of corresponding software solutions believe that generative AI will significantly improve their products. Salesforce, a leading provider of customer relationship management (CRM) solutions, is one example of a major player in this software market. The original AI assistant “Einstein Copilot” (Franzen 2023) has now been integrated into Salesforce’s new agent platform Agentforce (Salesforce 2024).

Cobus Greyling, the “Chief Evangelist” at Kore.ai (a provider of an AI agent platform), describes the evolution from handcrafted workflows to adaptive multi-agent, agentic AI (Greyling 2025b; Greyling 2025a). Sapkota, Roumeliotis, and Karkee (2025) made a critical distinction between “AI agents” and “agentic AI”. The paper presents a structured conceptual taxonomy, application mapping, and challenge analysis to clarify the differences in their design philosophies and capabilities. This study aims to provide a roadmap for developing robust, scalable, and explainable systems driven by AI agents and agentic AI.

Manus AI (Franzen 2025; Shen et al. 2025) is a general-purpose AI agent that represents a significant advancement in autonomous AI technology. Developed by a Chinese startup and introduced in early 2025, the tool is designed as a multi-agent system that combines several AI models to handle tasks independently. Devin AI (S. Wu 2024), developed by Cognition AI, is a cloud-hosted, fully autonomous coding agent. In April 2025, Cognition released Devin 2.0, featuring an integrated development environment designed for AI agent collaboration. He, Treude, and Lo (2025) conducted a survey exploring the current landscape of LLM-based multi-agent systems in software engineering.

Distributed AI has been an active research field for decades and produced important results regarding the realization of multi-agent systems that can now be adapted for agentic AI (Rossi et al. 2025). Yoav Shoham, a renowned expert in multi-agent systems and AI in general, recognizes the significant potential of AI agents. However, he, like several other authors (Koziolek et al. 2025; Mitchell et al. 2025; Huckins 2025), also acknowledges the substantial risks associated with fully autonomous software agents. Shoham (2025) emphasizes the importance of the semantic layer for reliably exchanging information between independently developed agents. Current protocols are not designed to ensure mutual understanding because they do not consider the meaning of transferred data.

Ehtesham et al. (2025) conducted a survey on interoperability protocols for LLM-based agents. The model context protocol (MCP), an open standard introduced by Anthropic (2024), connects AI systems with data sources, replacing fragmented integrations with a single protocol. The result is a simpler, more reliable way to give state-of-the-art models access to the data they need to produce better, more relevant responses. Google Cloud’s new open protocol, Agent2Agent (A2A), complements the MCP. Accenture is one of more than 50 technology partners providing support and contributions for A2A. Scott Alfieri from Accenture commented that “the multi-agent A2A protocol from Google Cloud is the bridge that will unite domain specific agents across diverse platforms to solve complex challenges, enabling seamless communication and collective intelligence for smarter and effective agentic solutions.” (Surapaneni, Jha, et al. 2025).



As LLM applications evolve from individual prompts to more complex and dynamic agentic systems, the generalized concept of “context engineering” has emerged. This term describes a more comprehensive and powerful approach that considers everything the model knows and perceives when generating a response (LangChain 2025). By consciously curating system instructions, short and long-term memory, retrieved knowledge, tool outputs, and guardrails within the limited context window, developers provide agents with the situational awareness needed to think, decide, and act autonomously across different tasks. While prompt engineering and RAG are valuable tools in this approach, they alone are insufficient for successful context engineering. Y. Ji (2025) discusses the important role of context engineering in creating the general-purpose AI agent, Manus, and shares insights on the best practices of context engineering.

2.4.2 Datasets for Code-Centric LLM Creation and Evaluation

There are datasets to be used for development and testing of AI-based software engineering. For pre-training models, a snapshot of selected public GitHub repositories is for example used by AlphaCode (M. Chen et al. 2021). This GitHub snapshot shows an unbalanced distribution of programming languages. For example, C++ provides 41% of the code files for training, while Rust, Scala, and Lua each provide less than 1%. DeepSeek Coder (Guo et al. 2024), for instance, combines code repositories with text repositories for the pre-training process.

The CodeContestV2 dataset (Y. Li et al. 2022) consists of competitive programming datasets mixed with problems, solutions, and test cases scraped from the Codeforces platform. It is used, for example, for fine-tuning the AlphaCode model.

The HumanEval test dataset (M. Chen et al. 2021) consists of programming tasks in which models must generate code snippets based on natural language descriptions. SWE-bench (Jimenez et al. 2024) is a dataset of 2,294 software development problems related to 12 Python repositories. The dataset is used to compare different methods or tools for programming assistance.

Data collections such as HumanEval and SWE-bench are used for evaluation and comparison of models. Competitive programming datasets from the two platforms LeetCode and GeeksforGeeks have also been used to evaluate the coding capabilities of LLMs. The Hugging Face platform (HuggingFace.co) allows organizations and individuals to share machine learning models and datasets, and to showcase their work. It is a premium source for state-of-the-art datasets.

The Software Heritage initiative (Abramatic, Di Cosmo, and Zacchiroli 2018; Di Cosmo 2018), based at Inria (the French Institute for Research in Computer Science and Automation), was founded to collect and preserve all publicly available source code. It is the world’s largest archive of its kind. StarCoder 2 (Lozhkov et al. 2024) is the first LLM family trained on Software Heritage data and provides code completion models.

In January 2025, Software Heritage started the Code Commons project, which aims to build a unified data platform that promotes transparency and accountability in AI development by offering pre-cleaned code collections with comprehensive metadata (Gent 2025). The goal is to create the world’s largest repository of ethically sourced code for AI training.



2.4.3 LLMs Adjusted for Software Engineering

AlphaCode (M. Chen et al. 2021) is pre-trained on GitHub and fine-tuned on CodeContestV2, a dedicated high-quality dataset. Often models are fine-tuned using reinforcement learning. DeepSeek for example uses group relative policy optimization (DeepSeek-AI et al. 2025). Rewards for the reinforcement learning are generated rule-based, through compilation of test cases or human feedback. Chain-of-thought (CoT) data can be used to generate step-by-step explanations as part of the answer of the model, either for fine-tuning or as few-shot examples in the prompt. For the retrieval of a solution, AlphaCode2 (Leblond et al. 2023) is used to generate a large number of potential solutions that are then filtered out, clustered and re-ranked by another trained ranking model.

Kessel and Atkinson (2025a) proposed the concept of “morescent” Generative AI, which are models trained on both the syntax and semantics of software, thus being “aware” of software behavior.

H. Ding et al. (2024) from the AWS AI Lab prepared a survey describing the technical components necessary for GenAI to have an optimal impact on software engineering. LLMs are used in a multitude of tasks related to software engineering, including core tasks such as requirement engineering (Section 3.2.1), coding (Section 3.3), and testing (Section 3.4.2), as well as operation and maintenance (Section 3.5).





2.5 Metrics for Evaluation

Given the need to use empirical evidence to make decisions in software engineering, measures are needed that assess software quantitatively and qualitatively (Ebert and Gustafson 2024). Quantitative metrics are calculated with dedicated formulas or measured directly with engineering systems. Qualitative assessment is typically derived from subjects, for example through semi-structured interviews, providing a subjective measure. Measures are required to evaluate all aspects of software engineering (Section 2.3), such as requirements, design, code, test, and maintenance.

Software Quality and DevOps

Software quality encompasses several perspectives, including the software engineering process, the final software product, and intermediate work products—that is, any artifact resulting from a defined process step. The concept of continuous improvement underlies software process monitoring, which requires the assessment of both process and product using a common, more holistic approach based on empirical methods. Green et al. (2024) distinguish three facets of developer productivity and efficiency—speed, ease, and quality—and explore what makes software quality difficult to define and measure. The authors identified four types of quality—process, code, system, and product—which influence each other.

DevOps metrics provide measurements designed to achieve the goal of accelerating speed and enabling continuous delivery, such as those developed by Google Cloud’s DevOps Research and Assessment (DORA) team (Forsgren and Kersten 2018). An overview of DevOps metrics and associated key performance indicators (KPIs) is provided by Amaro, Pereira, and Mira da Silva (2024). The study organizes metrics into four categories: business KPIs, change KPIs, operational KPIs, and cultural KPIs.

An extension to DORA is the SPACE framework (Forsgren, Storey, et al. 2021), which broadens measurements for evaluation into five dimensions: (1) satisfaction & well-being, (2) performance, (3) activity, (4) communication & collaboration, and (5) efficiency & flow.

Developer Experience (DevX) further builds on SPACE by focusing on developer workflow friction, tool usability, and emotional well-being (Noda et al. 2023). GitHub defines DevX as “examining how the juxtaposition of developers, processes, and tools positively or negatively affects software development” (Davis 2023). The concept introduces measurable indicators such as:

- Time lost to tool delays (e.g., CI pipeline wait times)
- Onboarding duration and perceived ease
- Cognitive overhead from context switching
- Developer sentiment and satisfaction feedback

DevX sharpens attention on experiential dimensions—bringing tool efficiency, developer flow, and satisfaction to the forefront. DORA’s Deployment Frequency and Lead Time for Changes map directly to SPACE’s Activity and Efficiency & Flow, while DORA’s Change Failure Rate relates to SPACE’s Performance dimension. SPACE, however, includes broader developer-centric dimensions—Satisfaction and Communication/Collaboration—that DORA doesn’t cover, illustrating SPACE’s more extensive scope (BlueOptima 2023).

The Flow framework (Kersten 2018) complements these with metrics like Flow Time, Flow Load, and Flow Efficiency—tracking end-to-end business value delivery.



Tangible Metrics for Software Development

Software development is a complex, creative activity shaped by technological and sociological factors; Jaspan and Green (2025) propose that evaluation models incorporate multiple outcomes, metrics, and methods. Rossi et al. (2025) highlight that current evaluation approaches for AI systems lack measures of usability and transparency.

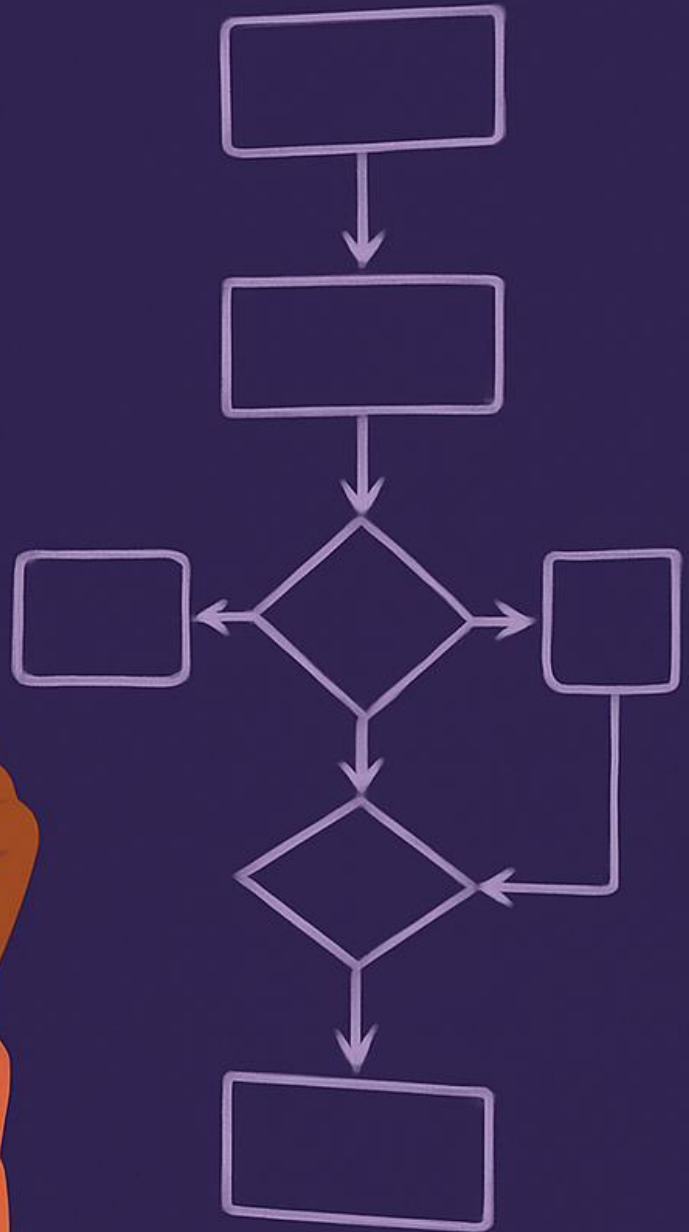
To ensure clarity and practical adoption, it is essential to anchor abstract dimensions in tangible metrics. Examples include:

- **Developer satisfaction:** measured via periodic team surveys (e.g., Likert-scale feedback)
- **Flow Efficiency:** calculated as the ratio of active development time to total cycle time using tool logs
- **Onboarding Experience:** measured via time-to-first-PR and new-hire feedback
- **CI Delay:** median queue time before build/test startup

These examples make evaluation frameworks actionable and contextually meaningful for practitioners.



3. Assessment of Core Tasks in Software Engineering





This section first considers the technology and infrastructure associated with Large Language Models (LLMs) for generative AI in software engineering. Subsequently, an evaluation of the application of LLM-based tools for core tasks within designated phases of the software development process will be conducted.

3.1 Infrastructure and Tools for AI-based Development

The landscape of software engineering tools has undergone a substantial transformation with the rise of generative AI technologies. Traditional development environments are being enhanced or replaced by AI-powered solutions that aim to boost developer productivity and efficiency, improve software quality, and accelerate software delivery throughout the entire software development life cycle.

Leading technology providers—including GitHub (Copilot), Microsoft (Azure OpenAI), Google (initially Codey, now integrated into the Gemini platform), and Anthropic (Claude Code)—have introduced AI assistants capable of code generation, review, and test automation. These tools can be categorized into several functional groups, such as language model-based assistants, code completion engines, automated bug detection systems, and test case generators. Most are deployed via cloud-based APIs and rely on scalable, high-performance infrastructure, typically utilizing GPUs or TPUs on platforms such as Azure, AWS, or Google Cloud.

3.1.1 The Landscape of AI Tooling for Software Development

The review by Y. Yang et al. (2022) shows that the use of deep neural networks for software development has been studied extensively for years. In the field of language technology, programming languages have played a crucial role in advancing natural language processing (NLP), as noted by Z. Zhang et al. (2024). Due to their precise syntactic and semantic structure, formal languages enable more reliable and automated evaluation of NLP techniques by testing whether the output code executes correctly. This foundational research has paved the way for recent advancements in code-generating large language models (LLMs).

GitHub Copilot: An AI-based Pair Programmer

M. Chen et al. (2021) introduced Codex, a language model based on an early version of OpenAI's GPT-3. Codex was fine-tuned using publicly available code from GitHub and became the foundation for the first release of GitHub Copilot. While generative AI in software tools existed before, GitHub Copilot distinguished itself through substantial productivity and efficiency gains. According to Greenstein (2024), Copilot improved the performance of intermediate-level developers by 20% to 40%, which is a significant breakthrough in a field where such gains are rare. Within two years, Copilot evolved from a prototype into a widely adopted assistant used by millions of developers (Harding and Matthew 2024). Today, it is considered the most widely used AI-powered developer tool globally (Benaich, Chalmers, et al. 2024).



Competitive Programming

Beyond providing practical development support, generative AI systems have demonstrated impressive capabilities in competitive programming tasks. These tasks typically require translating natural language descriptions into executable algorithms under time pressure. AlphaCode (Y. Li et al. 2022) was the first AI model to reach the median performance level of human competitors in these contests. Its successor, AlphaCode 2 (Leblond et al. 2023), powered by Google's Gemini model family, achieved significantly higher accuracy—solving 1.7 times more problems and outperforming 85% of human participants. Similarly, OpenAI et al. (2025) demonstrated that reinforcement learning can further improve LLM capabilities in complex reasoning. Their general-purpose model o3 achieved a performance rating comparable to that of elite human coders.

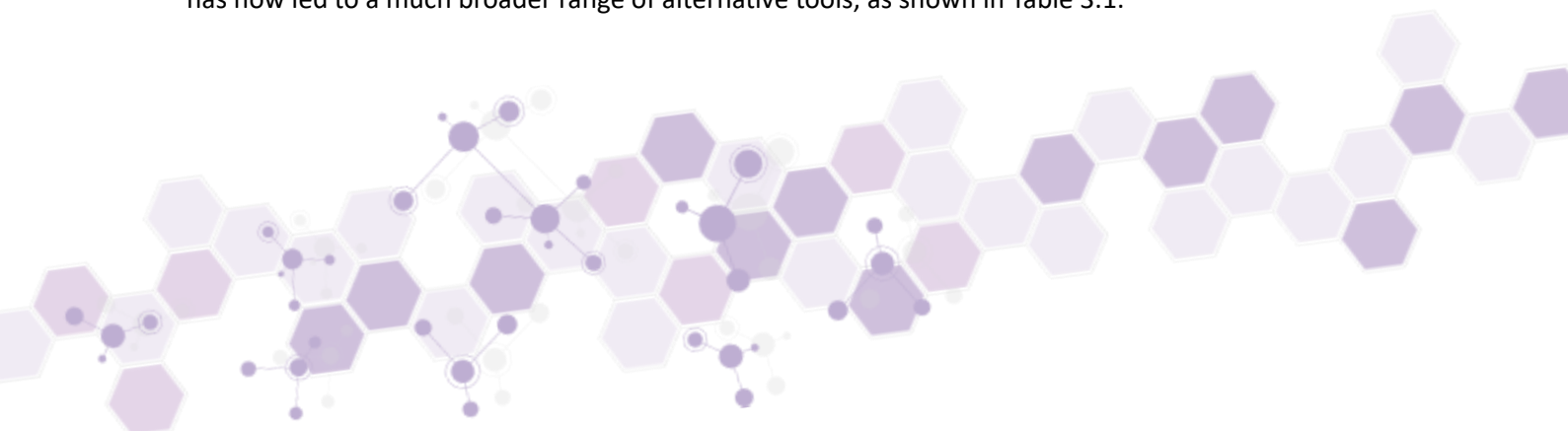
Castelvecchi (2025) reported that, for the first time, LLMs from DeepMind and OpenAI performed at the same level as gold medalist students in the International Mathematical Olympiad when it came to solving math problems. The continuously updated LiveCodeBench Pro benchmark (Zheng et al. 2025) indicates that LLMs still have difficulty with nuanced algorithmic reasoning and complex case analysis. Improvements in performance seem to be largely based on the precision of implementation and tool extension rather than superior reasoning. A team of Olympians comments on each problem in the algorithmic categories and performs a line-by-line analysis of the model's erroneous submissions. LiveCodeBench Pro provides detailed diagnoses to guide improvements in code-centric LLM reasoning and further close the gap to the level of human grandmasters.

AI Code Assistants

As adoption expands, industry benchmarks are emerging to evaluate the practical impact of AI coding assistants. Gartner defines AI code assistants as tools that help generate and analyze software code and configuration files. The Gartner Critical Capabilities report (Khandabattu et al. 2025) offers a comprehensive evaluation of such tools across different vendors. Crucially, Gartner emphasizes the need to establish baseline productivity and efficiency metrics prior to deployment. It recommends widely recognized frameworks such as DevOps Research and Assessment (DORA) and the SPACE framework (see Section 2.5) to guide the measurement of both developer efficiency and experience.

Agentic Workflows

Developing agentic AI applications that combine multiple agents into a workflow requires tools that facilitate this process. Initially, LangChain, LlamaIndex, and similar solutions received significant attention in the framework development field because of their pioneering approach. However, as developers' demands evolve and become more sophisticated, some have criticized these tools for their limited flexibility and high-level abstractions (Benaich, Chalmers, et al. 2024). Accelerated innovation has now led to a much broader range of alternative tools, as shown in Table 3.1.





A Selection of Agentic AI Frameworks

Name	Key Characteristics
LangChain	The first popular framework for building LLM-based applications, which was launched in 2022, became a de facto standard because of comprehensive vendor integration, cloud-vendor support, and third-party libraries integration.
LangGraph	The open-source framework from the LangChain team offers graph-based workflows, multi-agent capabilities, and native integration with LangChain. The LangGraph platform is a commercial solution for deploying agentic applications to production.
Autogen	The programming framework for building agentic AI agents and applications is developed by Microsoft. AutoGen provides a multi-agent conversation framework as a high-level abstraction and also supports cost and performance aware LLM inference APIs.
Semantic Kernel	The open-source development kit from Microsoft integrates LLMs and data stores to create product-scale GenAI solutions. Its Agent Framework provides a platform for AI agents, to incorporate agentic patterns into any application. The Process Framework offers an approach to optimize AI integration with business processes.
LlamaIndex	The initial focus was on data framework capabilities for LLM applications and has evolved to cover AI agents, document parsing & indexing, workflow, connectors-based integration, and other capabilities.
AutoGPT	The platform built by Significant Gravitas allows to create, deploy, and manage continuous AI agents that automate complex workflows, using LLMs from various vendors.
CrewAI	The focus of the multi-agent framework is to build and deploy workflow-based applications. Various LLM and cloud providers cooperate with CrewAI.
PydanticAI	The Pydantic team applies the simplicity of its FastAPI approach to GenAI application development and provides a model agnostic implementation, real-time observability, type-safety, graph-support, and dependency injection.
Spring AI	This project from the Spring team aims to provide a familiar and consistent Spring-style developer experience for building AI applications in Java. It simplifies the integration of LLMs and other AI capabilities into Spring applications.
Haystack	The open-source framework for building production-ready LLM applications, RAG pipelines, and complex search applications for enterprises. It has been built applying modular architecture principles combining technology from OpenAI, Chroma, Marqo, and other open-source projects, like Hugging Face Transformers or Elasticsearch.
MetaGPT	A multi-agent framework of AI agents that is built by researchers (Hong et al. 2024) to promote the idea of meta-programming.



Flowise	The open-source framework provides a drag-and-drop UI to build agents with customized LLM flows.
Camel-AI.org	An open-source multi-agent framework for building customizable agents that is inspired by CAMEL—Communicative Agents for “Mind” Exploration of Large Language Model Society (G. Li et al. 2023).
Firebase Genkit	A Google Firebase open-source framework for building AI-powered applications for Node.js and Go.
Agent Development Kit	ADK is a flexible and modular framework for developing and deploying AI agents. The open-source framework released by Google integrates with MCP and the A2A protocol (Surapaneni and Stephens 2025). While optimized for Gemini and the Google ecosystem, ADK is model-agnostic, deployment-agnostic, and is built for compatibility with other frameworks.
HF Smolagents	A Hugging Face open-source library to build powerful agents with simplicity, and first-class support for code agents.
Bee AI Framework	An open-source TypeScript library for building production-ready multi-agent systems.

Table 3.1: Selected frameworks for building AI applications with agentic flows.

Towards Agentic AI for Software Engineering

Smith (2025) discussed the best coding tools currently available, including the command line tool Claude Code and popular integrated development environments (IDEs) such as Cursor, Windsurf, and Visual Studio Code (VS Code), which integrates with GitHub Copilot. Innovative startups like Anysphere and Windsurf (formerly Codeium) have developed popular AI-focused products. Both the Anysphere Cursor and the Windsurf editor are VS Code forks. Windsurf calls its editor the first “agentic IDE” and has been acquired by Cognition AI (S. Wu 2025), the creators of the autonomous coding agent Devin. Another recent fork of the open-source VS Code editor is Amazon Kiro (Swaminathan and Singh 2025), which combines vibe coding with specification-driven development as an alternative to existing code-centric approaches. Specifications document all design decisions from concept to production because, in general, requirements are vague, making it difficult to determine whether the application meets them. Nearly all tool providers are gradually expanding their products in response to the obvious trend toward agentic AI.

Related Research on AI Adoption

The annual DORA developer survey (DeBellis et al. 2024) revealed that 75.9% of respondents already use AI in at least one daily work task. Chatbots were the most frequent interface (78.2%), followed by external web tools (73.9%) and AI features embedded in IDEs (72.9%). AI use via internal web interfaces (58.1%) and within CI/CD automation pipelines (50.2%) was reported less frequently.

To better understand where generative AI is currently applied in software engineering, the DORA report ranks twelve common development tasks based on the proportion of developers using AI for each task. These include tasks like code generation, documentation, testing, debugging, and others. This study will use the DORA task list as a guiding reference to assess the evolving role of generative AI across the software development life cycle.



3.1.2 Generative AI Infrastructure Considerations

As noted in recent research on computing infrastructure for large-scale generative models, organizations must strategically adapt their systems to support the demands of AI-based development tools (Lu et al. 2024). Unlike traditional development environments, generative AI tools require more dynamic, scalable, and resource-intensive infrastructure. These tools not only introduce new computational challenges, but also raise concerns related to integration, observability, and security.

Three core domains emerge when planning infrastructure for AI tooling: computing resources, system integration, and governance.

Computing Resources and Performance

Supporting large language models and AI-driven code assistants requires high-performance infrastructure:

- *Scalable compute environments*: Cloud-native platforms must be able to allocate GPUs or TPUs on demand to handle training or inference workloads (Flexential 2024).
- *Network throughput*: Fast, low-latency data pipelines are essential to avoid bottlenecks when integrating AI systems with developer environments.
- *Resource scheduling*: Efficient workload orchestration is needed to prioritize latency-sensitive development tasks.

Toolchain Integration and Developer Experience

Infrastructure must accommodate seamless interaction between generative AI tools and standard software engineering practices:

- *Version control system integration*: Compatibility with Git and GitHub/GitLab workflows is essential for collaborative AI-assisted coding.
- *CI/CD integration*: Generative AI outputs should feed directly into automated build and deployment pipelines (Lu et al. 2024).
- *Monitoring and logging*: Infrastructure should capture detailed telemetry on AI tool usage, performance, and failure points to support observability.

Security, Privacy, and Governance

Generative AI tools may process proprietary or sensitive code and data. This necessitates additional governance mechanisms:

- *Security protocols*: Identity and access management, encryption at rest and in transit, and audit trails must be enforced to safeguard code and model inputs (Flexential 2024).
- *Compliance requirements*: Infrastructure should be auditable and aligned with organizational and legal data protection standards (e.g., GDPR, SOC 2).
- *Model transparency*: Organizations must be able to explain and control how generative models are used internally, especially if integrated with internal repositories.



Overall, effective infrastructure planning ensures that AI-enhanced development workflows are not only performant but also reliable and secure. These considerations become increasingly important as generative AI tools are more deeply embedded into everyday engineering practices.

3.1.3 Evolution of Development Tools and Future Directions

Recent studies have demonstrated a paradigm shift in how developers interact with their tools. According to observational studies of ChatGPT usage in software development, developers are increasingly relying on AI assistants not just for code generation, but for a broader range of tasks including code explanation, debugging, and architectural decision-making (Khojah et al. 2024). This shift represents a fundamental change in the developer workflow, where AI tools serve as collaborative partners rather than simple code completion utilities. The evolution of software engineering tools continues to accelerate, with several key trends emerging.

Enhanced Context Awareness

Future tools are expected to better understand project-specific contexts, including:

- Architectural constraints
- Team coding standards
- Business domain requirements

Collaborative Features

Research indicates a trend toward more sophisticated collaboration between human developers and AI tools, with emphasis on:

- Interactive problem solving
- Knowledge transfer
- Context-aware suggestions

Cross-Platform Integration

Studies suggest that future tools will provide better integration across different development environments and platforms, reducing friction in the development process. The rapid evolution of AI-powered development tools represents both an opportunity and a challenge for the software engineering community. While these tools offer significant potential for improving productivity, efficiency, and code quality, their effective integration requires careful consideration of workflow, infrastructure, and quality assurance requirements.



3.2 Analysis and Design

Generative AI technologies, particularly large language models (LLMs), are transforming the analysis and design phases of software engineering. These models demonstrate strong capabilities in supporting tasks such as requirements engineering, user interface design, and architectural modeling. This section explores recent research and use cases illustrating how GenAI enhances productivity, efficiency, consistency, and automation across these tasks.

3.2.1 Requirement Engineering

Requirements engineering (RE) is a foundational activity in software development. It involves eliciting, structuring, and validating requirements to ensure that stakeholder needs are accurately translated into system specifications. Software engineers and managers, however, need the right tools to support the requirements management process. One leading tool is IBM Engineering Requirements Management DOORS (“DOORS” 2017). The DOORS (Dynamic Object-Oriented Requirements System) product line offers an enterprise-wide, multi-platform solution for managing requirements.

AI-Based Tools for Software Requirements

AI-based tools provide extra support when dealing with the requirements of a software product. Microsoft Copilot 365, for example, makes it easier to maintain and use associated documents.

The DFKI AlxSystems platform provides practical AI-driven tools for requirements engineering (Saleem, Asim, van Elst, and Dengel 2023; Saleem, Asim, van Elst, Junker, et al. 2024; Saleem, Asim, and Dengel 2025). Saleem, Asim, van Elst, and Dengel (2025) conducted a comprehensive case study investigating the potential of ChatGPT and Gemini across four public benchmark datasets of requirements engineering tasks. Both models performed worse than existing task-specific machine learning (ML) predictors and traditional language models.

Other recent studies, however, demonstrate that LLMs can automate and augment RE workflows, offering improvements in classification accuracy, contradiction detection, quality assurance, and even full life cycle development based on natural language input.

Classification of Functional and Non-Functional Requirements

One of the most immediate applications of LLMs is the classification of natural language requirements. Hey et al. (2020) fine-tuned BERT into a model named NoRBERT, achieving superior performance in classifying functional and non-functional requirements. Notably, their model generalized well across both seen and unseen data, demonstrating the effectiveness of transfer learning in this domain.

MLR-Predictor (Saleem, Asim, van Elst, Junker, et al. 2024), which is part of the DFKI AlxSystems platform, is a computational framework for multi-label classification of requirements. In addition, the hybrid, specialized FNReq-Net (Saleem, Asim, van Elst, and Dengel 2023) framework allows to distinguish between functional and non-functional requirements.



Contradiction Detection and Clarification

LLMs also play a role in identifying and clarifying ambiguities and contradictions in requirements documents. A prominent example is the ALICE system (Automated Logic for Identifying Contradictions in Engineering), introduced by Gärtner and Göhlich (2024a). ALICE uses a multi-phase approach involving:

- Identification of conditional statements
- Extraction of pseudo-grammatical structures
- Contradiction detection using LLMs enhanced by formal logic

Gärtner and Göhlich (2024b) report that ALICE achieved 99% accuracy and 60% recall, significantly outperforming LLM-only baselines. These results demonstrate the value of combining symbolic reasoning with language models for robust requirement verification.

PassionNet (Saleem, Asim, and Dengel 2025) from DFKI is a multi-model ML framework for the automatic identification of conflicting and duplicate requirements.

Requirements Quality Assessment and Validation

Another research stream focuses on automating the quality evaluation of requirements. According to Porter, DeFranco, and Laplante (2025), LLMs can be integrated into quality analysis pipelines that evaluate requirements along dimensions such as consistency, completeness, and feasibility. These automated assessments help reduce manual overhead and promote early error detection.

End-to-end Approaches: From Requirements to Full Systems

Some researchers envision a more radical transformation of the RE process. B. Wei (2024) explores how LLMs can translate high-level requirements directly into code, marking a shift toward fully automated pipelines. Similarly, Arora, Grundy, and Abdelrazek (2023) assess how LLMs can enhance requirements elicitation, traceability, and refinement.

Perhaps most notably, Robinson et al. (2025) argue that generative AI could eventually allow end-users to manage the entire software life cycle—starting from requirements expressed not only in natural language but also through multimodal inputs such as diagrams, images, video, or audio. This would mark a shift from requirements engineering as a technical activity toward a more human-centered, conversational design process.

These developments illustrate how GenAI is reshaping requirements engineering—not simply by automating classification, but by introducing intelligent, context-aware, and increasingly creative support for managing the early stages of software development.





3.2.2 UI/UX Design

Large Language Models are increasingly used to accelerate and enhance UI/UX design workflows. These models can generate user interface prototypes based on natural language requirements, assist with layout suggestions, and enable designers to quickly iterate based on user feedback. Importantly, LLMs also offer new modes of creativity support, ethical concern monitoring, and mobile-first personalization. Based on his early case studies, Nielsen (2023) predicted substantial productivity and efficiency increases from AI-assisted UX work.

Jialiang Wei et al. (2025) propose three complementary AI approaches for supporting UI designers: (1) LLM-based prompting, which enables natural language-based generation of layout ideas; (2) vision-language retrieval, which surfaces diverse design inspirations based on multimodal inputs; and (3) diffusion-based UI generation, which leverages generative image models to produce stylistically diverse mockups. Together, these methods stimulate creativity, expand ideation possibilities, and support early-stage UI development.

However, the use of generative AI in interface creation also introduces significant ethical risks. Krauß et al. (2025) demonstrate that AI models such as ChatGPT, Gemini, and Claude can autonomously incorporate deceptive user interface components—known as dark patterns—into generated HTML or app designs. Alarming, such elements may be introduced even from neutral prompts, without any notification to the developer about the legal or ethical implications. This underscores the need for responsible prompt design and validation during AI-assisted UI generation.

D. Chen et al. (2025) explore the integration of LLMs in mobile-native design workflows. Their roadmap outlines how generative AI can be embedded into mobile development environments to enable real-time personalization, voice-guided interaction design, and conversational UI building. For example, LLMs can assist in tailoring interfaces to individual user preferences, automating onboarding flows, and interpreting spoken feedback to dynamically adjust app layouts. This marks a shift toward mobile UX systems with embedded intelligence and contextual responsiveness.

Beyond academic literature, industry evaluations show a mixed landscape. Petridis, Terry, and Cai (2024) present PromptInfuser, a Figma plugin that tightly couples AI-generated suggestions with designer control, revealing new collaboration patterns. At the same time, Sponheim and Brown (2024) from the Nielsen Norman Group caution that many AI-UX tools remain limited in usability, accuracy, and transparency—reinforcing the importance of human oversight in production workflows.

Figma AI (Rasmussen 2024) is a set of features designed to increase efficiency and creativity. Figma's generative AI features, which produce designs, are based on third-party, off-the-shelf AI models.

Overall, while generative AI introduces speed and breadth to the UI/UX process, its successful integration depends on ethically grounded design practices, robust human–AI collaboration, and tool maturity.





3.2.3 Software Design

Generative AI is also reshaping the domain of software design, particularly in activities involving formal modeling, architectural decision-making, and automated analysis. Tools powered by large language models (LLMs) are now being explored for their ability to assist in generating software models, supporting model-driven engineering (MDE), and integrating design knowledge into intelligent tooling.

One major area of interest is the automatic generation of software models, especially using standardized modeling languages such as UML. Cámara et al. (2023) provide an experience report evaluating ChatGPT's capabilities in producing UML diagrams based on textual input. They highlight both promising initial results and key limitations, such as ambiguity in output structure and difficulty handling complex relationships. Similarly, Combemale, Gray, and Rumpe (2023) analyze how LLMs perform in various software modeling scenarios, revealing potential for automation but emphasizing the need for model validation and refinement.

From a broader model-driven engineering perspective, Burgueño et al. (2025) reflect on past automation efforts and propose a roadmap for incorporating LLMs into future MDE workflows. They argue that GenAI could lower the barrier to entry for modeling tools and extend the reach of MDE into more agile development contexts.

Business processes are a particularly important area of application for model-based engineering. Fettke and Di Francescomarino (2025) conducted a survey that provided a classification and analysis of the field of business process management (BPM) and AI. Kampik et al. (2025) propose the large processmodel (LPM) as a conceptual framework. LPMs combine large language models (LLMs) with knowledge-based and automated reasoning approaches for BPM. The authors discuss the potential and limitations of LLMs and foundational models for BPM. In this context, DFKI is also working on the development of new, powerful modeling formalisms (Fettke and Reisig 2024).

On the architectural level, Ozkaya (2023) investigates whether explicit architecture knowledge can guide LLMs in producing more structured and maintainable design solutions. Her work suggests that embedding software architecture patterns into prompts or training data may improve the alignment between generated code and system-level constraints.

Finally, data analysis is increasingly viewed as a cross-cutting concern in software design. According to the DORA developer report (DeBellis et al. 2024), data analysis ranks eighth among the top twelve AI-assisted development tasks. The advent of machine learning operations (MLOps) marks a turning point in the implementation of machine learning (ML) for data analysis (C. K. Tantithamthavorn et al. 2025). It enables organizations to scale AI/ML systems more effectively. The MLOps approach is designed to facilitate continuous delivery and ensure the consistent, reliable operation of models in production environments. Within the design phase, LLMs can assist with traceability, metric-based model evaluation, and performance-related tradeoff analysis—offering a more integrated, feedback-driven approach to design decisions.

Together, these developments signal a future in which LLMs not only generate design artifacts but also participate in high-level reasoning and feedback loops, potentially evolving into collaborative agents for model-driven development.



3.3 Implementation

This section looks at key aspects of code writing, the highest ranked task on the list of the 12 most important activities from the developer survey by DeBellis et al. (2024). When used effectively as a collaborative tool, generative AI can reduce the time spent on time-consuming programming tasks and thus improve the efficiency of developers.

Coding assistance tools, like for example GitHub Copilot (Dohmke 2022), Amazon Q Developer—formerly CodeWhisperer (Barr 2022), and Gemini Code Assist (Karunaratne 2025), which evolved from Google Duet AI (Weingartner and Seroter 2023), offer features such as autocomplete, code snippet generation, and code suggestions.

BLEU (Papineni et al. 2002) and ROUGE (Lin 2004) are traditional evaluation metrics that measure the similarity between generated and (human-produced) reference text. These metrics work well for natural language, but they can be inaccurate for code. CodeBLEU was developed for the automatic evaluation of code synthesis. Ren et al. (2020) proposed the use of weighted n-gram and syntactic AST matches to measure grammatical correctness and a semantic data-flow match to calculate logical correctness. The CodeBLEU metric has been shown to correlate better with human evaluation than BLEU and accuracy metrics.

3.3.1 Code Generation

Code generation from requirements or through the targeted use of prompts enables developers to quickly and accurately create code that meets specific requirements. This not only speeds up the development process, but also improves code quality as the generated code snippets are usually based on proven patterns and best practices (Liu et al. 2024). Most IDEs include code generation and retrieval functionality.

Python and the PyCharm IDE were used as examples in the early empirical study by Xu, Vasilescu, and Neubig (2022). The study investigated the remaining gaps and challenges of using natural language technology for code generation inside an integrated development environment.

W. Hou and Z. Ji (2025) evaluated the code generation performance of seven leading LLMs using test data from LeetCode and GeeksforGeeks coding contests. They showed that in the test data evaluated in the study, GPT-4 is the best performing LLM at generating programming code, outperforming other LLMs such as Gemini Ultra and Claude 2. Using an optimal prompt strategy, GPT-4 outperforms 85 percent of human participants in a competitive environment, many of whom are students and professionals with moderate programming experience.

Early results from studies of AI tools in full-stack development demonstrate capabilities across multiple layers of the application stack (Abrahamsson et al. 2024). These tools show particular strength in:

- Frontend component generation
- API design and implementation
- Database schema definition
- Infrastructure-as-code generation

AI-based software development can reduce initial code writing time by 40%, especially for boilerplate code and repetitive logic (Full Scale 2025).



Due to their limited proficiency in utilizing domain-specific libraries, LLMs can exhibit sub-optimal performance in generating domain-specific code. Gu et al. (2025) introduce a new code generation approach called DomCoder that uses different strategies for incorporating domain knowledge, namely, external knowledge inquirer, chain-of-thought prompting, and chain-of-thought fine-tuning. Experimental results show that these three strategies improve the effectiveness of domain-specific code generation under certain settings. Lyu et al. (2025) provide a comprehensive review of the latest advancements in code generation and consider various datasets for evaluation.

AlphaCode (Y. Li et al. 2022) was the first system to achieve automated code generation at a competition level. AlphaEvolve (Novikov et al. 2025), also developed by Google DeepMind, is an evolutionary coding agent that provides enhanced capabilities for highly challenging tasks. It builds on previous research (Romera-Paredes et al. 2024) on using program search with large language models in automated code generation. AlphaEvolve orchestrates an autonomous pipeline of LLMs to improve an algorithm iteratively by making direct changes to the code. The results presented in the report illustrate that this feedback-driven, evolutionary approach can lead to new scientific and practical discoveries.

3.3.2 Code Suggestion

The generative AI-powered models, which are trained on extensive codebases, facilitate intelligent code completion and suggestion (J. Chen et al. 2024; Nguyen and Nadi 2022). Rather than searching through huge codebases or extensive documentation and forum posts, developers can use natural language prompts to search for code snippets in codebases. Tools such as GitHub Copilot or Tabnine facilitate this process, they are capable of analyzing the surrounding code and comments to generate context-sensitive suggestions, bug fixes, and code refactoring (G. Kim and Yegge 2025). Moreover, retrieval-augmented generation (RAG) can also be employed in this context for interaction with a vast library of code snippets and documentation, as well as for code suggestion retrieval.

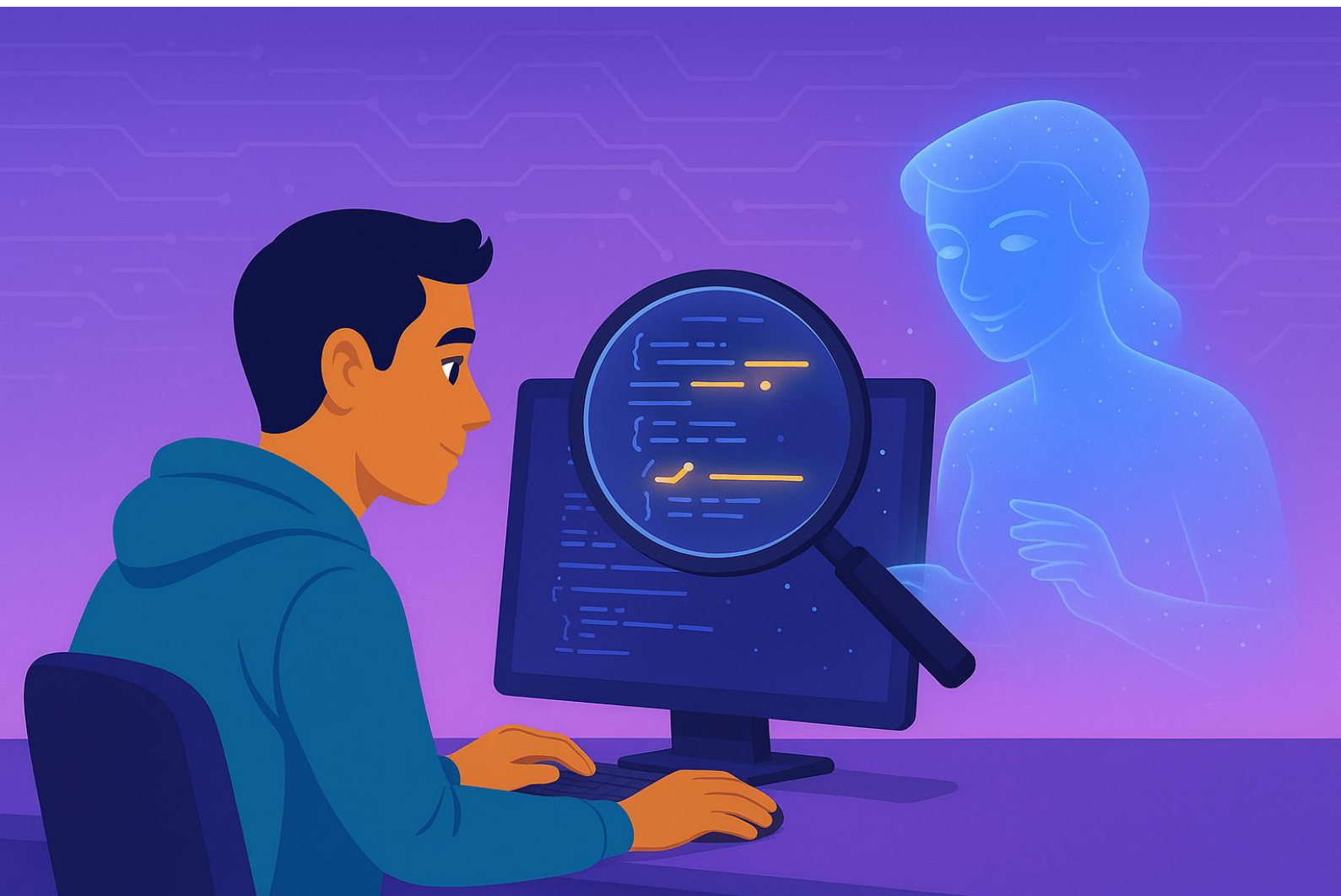
Relevantz (2024) report better code quality with approximately 30%–40% improvement in code review and correction workflows through AI-driven suggestions.





3.3.3 Code Explanation

The latest generative AI models incorporate reasoning and problem-solving capabilities for software development practices (OpenAI et al. 2025; Y. Yu et al. 2025; C. Tantithamthavorn, Cito, et al. 2023), particularly through the implementation of agentic AI (Fourney et al. 2024). The objective of these models is to leverage the understanding of code patterns, syntax, and semantics afforded by LLMs to make their decisions and processes understandable to humans and to assist developers in a range of software development tasks. These tasks range from code generation and automatic documentation generation to interactive code reasoning and explainability. (DeepSeek for example, generates step-by-step explanations of their reasoning process, allowing users to understand why the AI arrived at a particular conclusion.) As Borg et al. (2023) pointed out, explainability needs to be investigated, to provide genuine, non-intrusive explanations tailored to the developer. Code explainability and reasoning are also important issues in the DORA framework. As highlighted in the top 12 of the 2024 DORA report (DeBellis et al. 2024), such models have the potential to facilitate the comprehension of unfamiliar code, including the ability to summarize code functionality, generate adapted documentation, and assist developers in better understanding and further developing existing or legacy code. Significantly reducing documentation time by automatically generating code comments, summaries, and usage examples is an important process improvement. DeBellis et al. (2024) ranked explaining unfamiliar code third and documenting code fifth in their list of the top 12 software engineering tasks.





3.4 Integration and Testing

GenAI also impacts integration and testing tasks. For instance, it improves test case generation, enables continuous testing, and helps increasing delivery frequency and reliability. It also ensures quality assurance by using MLOps workflows and the DORA approach. The next two subsections cover these topics.

3.4.1 Software Integration

Y. Zhang et al. (2024) examined the application of generative AI in container orchestration, aiming to systematically characterize potential quality issues in ChatGPT-generated software artifacts. Their empirical study revealed that 35.8% of Kubernetes manifests generated by ChatGPT contained at least one instance of a “smell.” Based on these findings, the authors recommend that practitioners apply suitable quality assurance activities to ChatGPT-generated Kubernetes manifests before using them for container orchestration.

Lu et al. (2024) describe an emerging AI-native computing paradigm at the intersection of large generative AI models and cloud-native computing architectures. Large-model-as-a-service (LMaaS) is a cloud-based approach that could bridge the gap between advanced AI and real-world applications by offering efficient, scalable, and accessible deployment mechanisms. However, according to the authors, horizontal scaling-out scenarios have yet to become focuses of research and production.

Patel et al. (2025) investigated the complexities of integrating large language models (LLMs) into software products, with a focus on the challenges of determining their release readiness. The article synthesizes a release checklist for generative AI-based products to enable the future automation of release-readiness evaluation steps.

MLOps (machine learning operations) enables organizations to scale AI/ML systems more effectively and ensure continuous delivery. MLOps also helps maintain model performance and trustworthiness in production environments. According to C. K. Tantithamthavorn et al. (2025), MLOps combines machine learning (ML) and software engineering principles into a set of practices and tools that facilitate the development, deployment, and maintenance of ML models in production environments. MLOps aims to bridge the gap between data science and operations, enabling organizations to deliver reliable, scalable, and efficient ML solutions. The MLOps lifecycle is a series of stages that integrate ML development and operations to efficiently deploy, manage, and continuously improve ML models.

3.4.2 Testing

Research on AI-generated code quality indicates that while these tools can produce functional code, they require careful validation. Studies show that AI-generated code can sometimes introduce subtle bugs or security vulnerabilities that necessitate human oversight (Vaidya and Asif 2023).

As discussed in Ferrara, Arceri, and Cortesi (2024), formal program verification methods will also have to deal with AI-generated code. Although the use of third-party code in software projects is a common practice, verifying software by focusing on the external code is still an open challenge in the field of software verification.

Tricentis was an early participant in the AI-augmented software testing market (Tricentis 2021).



Creating Test Scripts

Based on their developer survey, DeBellis et al. (2024) ranked test writing sixth on the list of the top 12 tasks for AI-based automation. Bhatia et al. (2024) compared ChatGPT with PyPenguin, an existing unit test generator for Python programs, in terms of their ability to automatically create unit test scripts. The generated unit tests were evaluated based on criteria such as coverage, correctness, and readability. The results of the study suggest that using a combination of tools could improve automated unit testing performance. The GPT-style language model CAT-LM (Aligned Code And Tests Language Model), which is described in Rao et al. (2023), aims for a novel test generation tool that explicitly considers the mapping between code and test files when available.

Using AI-suggested unit and integration tests, encompassing edge cases that are often overlooked by humans, results in enhanced test coverage and quality (PwC 2024; Full Scale 2025).

S. Yu et al. (2023) investigated the application of LLMs in the domain of mobile application test script generation. The study's primary focus was on the generation of scenario-based tests and the migration of tests across platforms and applications using ChatGPT. It also identified the current limitations of this technology, offering valuable insights for future research.

Software Security Analysis

A. Ding et al. (2024) provide a detailed review of the state-of-the-art in generative AI for software security analysis. They divide software security analysis tasks into four primary categories according to the software lifecycle: risk analysis, code auditing, software security test, and binary analysis. DeBellis et al. (2024) ranked security analysis 10th out of 12 tasks in their developer survey. Natella et al. (2024) examined the security uses and misuses of AI code generators and reported on their experimental evaluation of such tools for security purposes. The authors conclude that cybersecurity professionals must embrace AI code generators to prevent attacks more efficiently. Their experimental results indicate that choosing the most appropriate generator is crucial. The experiments also demonstrated the importance of fine-tuning the models for security-oriented applications.

Dissecting low-level malware behaviors and generating human-readable reports is time-consuming and requires expertise in systems and cybersecurity. Sun et al. (2024) use a combination of dynamic analysis and generative AI to synthesize reports on malware that provide detailed technical insights and explain the intentions of the malware.

Pearce et al. (2025) systematically analyzed the conditions under which the technical preview of GitHub Copilot may recommend insecure code, focusing on high-risk vulnerabilities. As part of the study, a range of scenarios was created to represent common programming tasks. A review of the code generated by GitHub Copilot for these scenarios revealed that approximately 40% of the programs were vulnerable. Böhme et al. (2025) survey recent advances in software security, discuss open challenges and opportunities, and present a vision for the future of software security analysis.





Ensuring the reliability and security of AI-generated code necessitates a multifaceted quality assurance (QA) approach. Recent studies and industry practices emphasize the following key components:

- **Systematic validation of AI-generated code**

Implementing structured validation frameworks is crucial to assess the correctness and safety of AI-produced code. Techniques such as A/B—or differential—testing and comparative analysis across multiple AI-generated outputs can enhance reliability (Kessel and Atkinson 2025b).

- **Integration of automated testing frameworks**

Incorporating AI-driven testing tools can expedite the identification of defects and vulnerabilities. These tools facilitate continuous testing and validation throughout the development life cycle (Aleti 2023).

- **Development of AI-specific code review practices**

Further details concerning the use of AI-based tools in the context of code reviews are provided in the following Section 3.5.1.

- **Implementation of security scanning protocols**

AI-generated code can introduce unforeseen security risks. Employing advanced static and dynamic analysis tools helps in early detection and mitigation of potential vulnerabilities (Masood 2025).

Collectively, these strategies contribute to a robust QA process, ensuring that AI-assisted software development maintains high standards of quality and security.





3.5 Operations and Maintenance

The process of continuous release to production requires stringent quality assurance measures to ensure compliance with all service-level agreements. The operations domain provides valuable input for improving the software, such as feedback from technical product support and comprehensive application monitoring. This feedback can then be incorporated into the next iteration of the software development cycle.

Atlassian Intelligence (Trueman 2023; Valliani 2025b; Valliani 2025a) and GitLab Duo (DeSanto 2023) are prominent GenAI tools that significantly facilitate software deployment, release, operations, and maintenance. They efficiently manage deployment pipelines and integrate with various CI/CD tools, optimizing the deployment process.

3.5.1 Code Review

The DeBellis et al. (2024) list of the top 12 tasks resulting from their developer survey placed code review in ninth place. Davila, Melegati, and Wiese (2024) examined the existing discussions about generative AI-based tools in the context of code reviews. Their study provides evidence from practitioners who have explored this new technology. For developers, this new family of tools can positively impact code review efficiency and reduce reviewers' workload by flagging problems early and suggesting solutions. For example, ChatGPT can generate explanations to help understand code changes, enabling self-review. Using GitHub Copilot to write and review pull requests (PRs) can reduce review time and increase acceptance.

Y. Yu et al. (2025) emphasize the importance of improving the comprehensibility of automated code review, in addition to its accuracy. They introduce Carllm (Comprehensibility of Automated Code Review using Large Language Models), a fine-tuned LLM capable of improving both accuracy and comprehensibility, surpassing state-of-the-art pre-trained models and general LLMs.

Even at an early stage, C. Tantithamthavorn, Jiarpakdee, and Grundy (2021) addressed the need to develop AI-specific code review practices. They discussed the “why” and “how” of explainable and actionable software analytics for AI-generated code. Using Explainable AI techniques in software engineering (C. Tantithamthavorn, Cito, et al. 2023) is also essential for adapting review practices to the unique characteristics of AI outputs.

3.5.2 Program Repair

Debugging code placed seventh on the DeBellis et al. (2024) catalog of the top 12 AI-supported activities stemming from their developer survey. Lyu et al. (2025) discuss how advances in software engineering, such as program repair and analysis, can enable automatic programming.

Among early models of program repair are SequenceR (Z. Chen et al. 2021), Recoder (Zhu et al. 2021) and the initial Codex LLM (M. Chen et al. 2021). SequenceR uses sequence-to-sequence learning to create suggestions for corrected code, similar to a neural machine translation model that translates English into French, for example. Recoder generates edit sequences to fix a bug and relies on transformers incorporating abstract syntax trees, thereby limiting the search space to syntactically correct code, evaluated on dedicated bug databases—such as the Defects4J benchmark set (Just, Jalali, and Ernst 2014). Codex fixes the code directly. The LLM was fine-tuned using code from GitHub, including comments and changes to the code.



Modern AI systems can mirror human-style debugging processes through iterative reasoning and error resolution. Microsoft Research's "debug gym" environment (Yuan et al. 2025) shows that reinforcement-trained agents can autonomously trace, interpret, and patch faulty code based on output logs. Moreover, integrated AI tools like GitHub Copilot Chat allow developers to directly query the rationale behind bugs and receive inline patch recommendations, reducing time-to-resolution by up to 37% in team settings (Ya Gao 2024).

Pasquale et al. (2025) discussed the challenges of using LLMs for automated code generation and program repair in an expert round. This led to the question of whether LLMs will alleviate essential complexity or exacerbate accidental complexity in code repair tasks.

Recent advances in multimodal AI (e.g., Google Gemini, Amazon Kiro) have enabled systems to interpret screenshots, natural language voice prompts, and log files in real time. This facilitates a more natural human-AI collaboration, especially in debugging or onboarding workflows. For example, Gemini can extract stack trace intent from annotated images, while Kiro auto-generates repair code from spoken task descriptions (Kilpatrick 2024; E. Kim 2025).

3.5.3 Code Maintenance

Software maintenance encompasses the process of modifying, updating, and improving existing software code following its initial release. This section focuses on two selected key topics as part of our assessment of the use of generative AI in software development.

Code Optimization

One element of maintenance work is code optimization, which ranked fourth among the twelve tasks covered in the developer survey by DeBellis et al. (2024). Qiu et al. (2025) conducted a comparative analysis of the current state of AI-assisted programming in 2024 and made projections for 2030. The authors note that developers often rely on the copy-and-paste method while programming. These developers do not place enough importance on code optimization. Evaluating developers based on their individual productivity and efficiency can encourage them to copy code verbatim instead of refactoring old code. Additionally, developers often overestimate the capabilities of tools like GitHub Copilot and delay the verification of generated code.

Support for challenging software development tasks, such as code optimization and refactoring, remains limited. Currently available tools frequently struggle with code optimization, which can result in misleading errors and misunderstandings when developers fail to provide the necessary context. The comparison of the work routines of developers in 2024 and 2030 predicts that future tools will fully support developers in optimizing code, favoring the use of existing code and aligning code with comments. The vision is that AI will optimize code by monitoring coding in real time and suggesting alternatives.

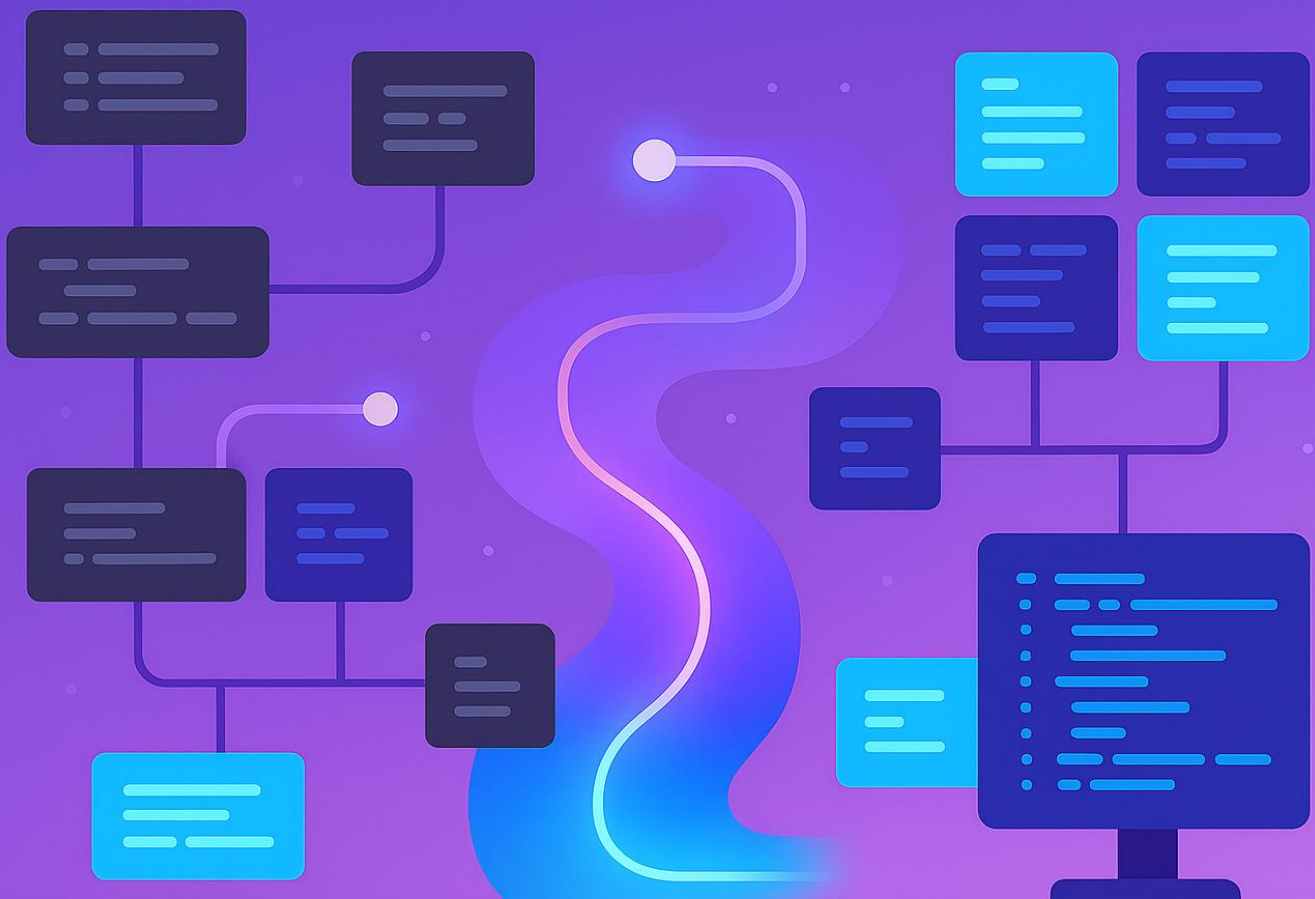
Code churn, also known as rework, is a metric that indicates how often a piece of code (e.g., a file, class, or function) is edited. While code that is rewritten or deleted shortly after being written is normal, too much of it can lead to decreased productivity and efficiency as well as quality issues. GitClear's AI code quality research reveals troubling trends regarding maintainability (Harding and Matthew 2024; Harding 2025). Investigations of highly structured code change data indicate that AI coding assistants significantly contribute to the degradation of code quality.



Language Migration and Codebase Modernization

DeBellis et al. (2024) ranked language migration 11th out of 12 tasks in their developer survey. Codebase modernization was the 12th-ranked task. The current state of contemporary software modernization is summarized by Assunção et al. (2025). In their tutorial report, Agarwal et al. (2024) describe the transition from non-AI to generative AI approaches. API migration is an essential step in migrating code between libraries or programming languages. Zhou et al. (2023) propose a hybrid approach combining small API mapping models with large language models (LLMs).

As stated in the experience report by Nikolov et al. (2025), generative AI-powered workflows enable Google to migrate code more efficiently and maintain its codebase more effectively. A combination of techniques based on abstract syntax trees (AST), heuristics, and LLMs has proven to be well suited, as described in the case studies in the paper.





3.6 AI-based Software Development Tools for Specific Engineering Tasks: Accenture Practice

In this section, we report on two studies by Accenture on the adoption of GenAI tools. In addition, we introduce the GenWizard generative AI enterprise platform.

3.6.1 Study on Adoption of GitHub Copilot

A study by Accenture and GitHub (Ya Gao 2024) assessed the adoption of a AI-based coding assistant. The participants included developers that are occupied with different tasks such as engineering, design and testing. The results are as follows: Developers accepted around 30% suggestions by GitHub Copilot. 90% of the developers committed code suggested by GitHub Copilot. 51% of the participants found GitHub Copilot extremely useful. 95% of developers reported enjoying coding more when utilizing GitHub Copilot. 95% of the participants felt more fulfilled with their jobs, when using GitHub Copilot.

3.6.2 Study on AI Use for Different Software Engineering Tasks

Another informal, internal survey was conducted by Accenture among its employees in February 2025. The cohort was composed of developers, testers, technical architects, business analysts and other profiles. As a result, 89% out of 132 respondents answered in a forced-choice test that their software engineering results improve when using a GenAI Tool for software engineering. Table 3.2 displays for what tasks in software engineering GenAI tools are used, according to the same survey. It appears that GenAI tools are most frequently used for code generation and document creation.

Tasks and GenAI Usage Percentage			
Requirement analysis	System design	Code generation	Test case generation
22%	16%	53%	22%
Bug diagnosis and debugging	Code review	Continuous integration and deployment (CI/CD)	Document creation
20%	16%	11%	45%

Table 3.2: Percentage of subjects that use GenAI for different tasks in software engineering among 134 respondents in a forced choice test.



3.6.3 Accenture GenWizard Tool

GenWizard is Accenture's full-suite generative AI enterprise platform that enables scaling of generative AI in technology delivery across application and infrastructure management, application and data modernization, and software and platform development. Figure 3.1 provides an architecture overview.

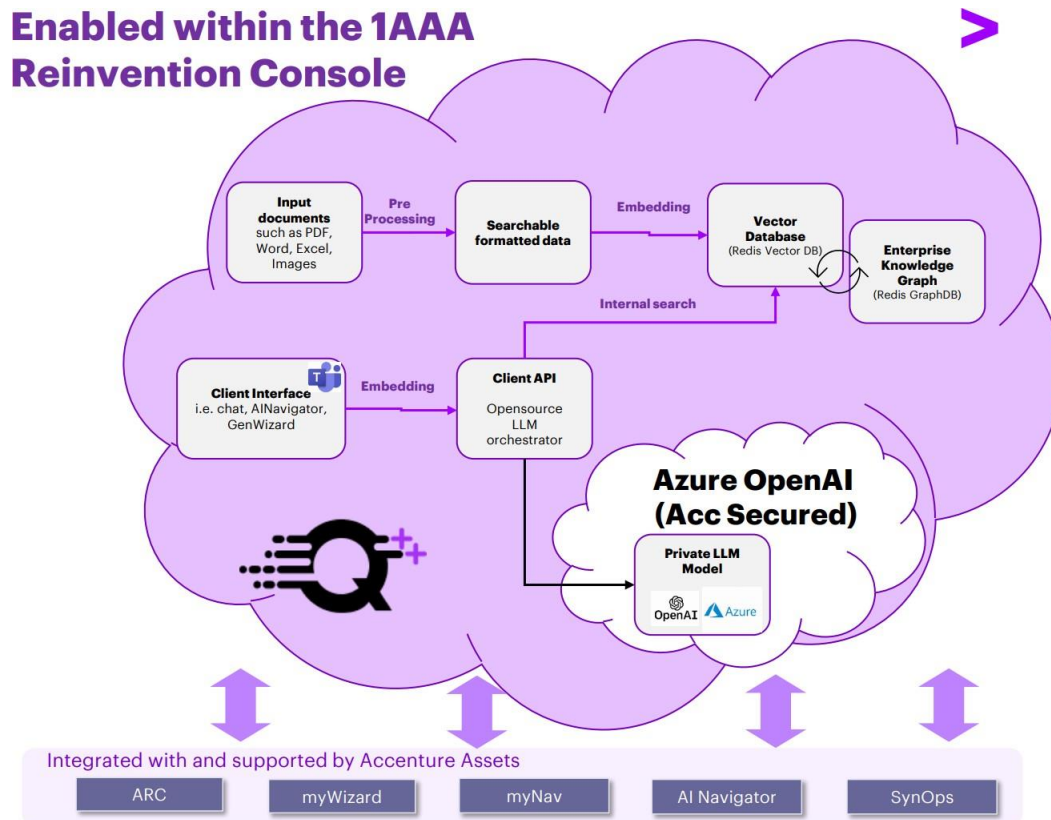
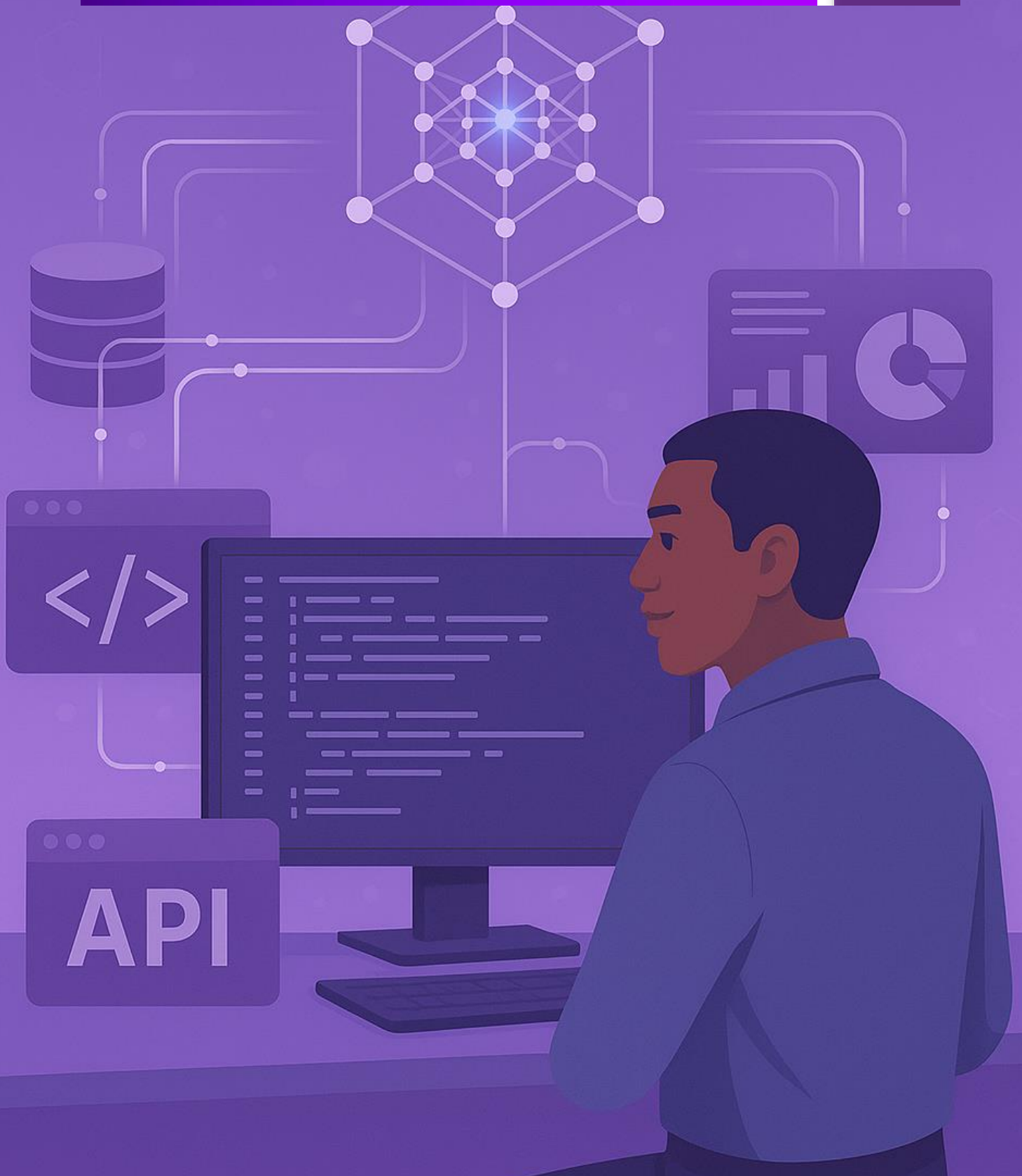


Figure 3.1: Architecture of GenWizard: Interaction of vector database, knowledge graph and LLM.

GenAI tools are transforming the way many people work. People in various roles in the software development process, from business analysts and DevOps engineers to front-end and back-end developers, project managers, and software architects, can benefit from GenAI.



4. Integration of Generative AI Across the Software Development Process





This section examines the cross-task aspects of using generative AI in software engineering and the associated effects on the entire software development process. Furthermore, the potential of an LLM-supported approach is demonstrated using a practical example. Finally, the discussion turns to the adoption of generative AI.

4.1 Large Language Models as General-Purpose Tools in Software Engineering

Figure 4.1 illustrates DFKI's vision (Wahlster 2023) for a new era of software engineering. The concept of "Software 3.0" has now become mainstream among new AI entrepreneurs (Karpathy 2025b).

The Third Generation of Software Engineers

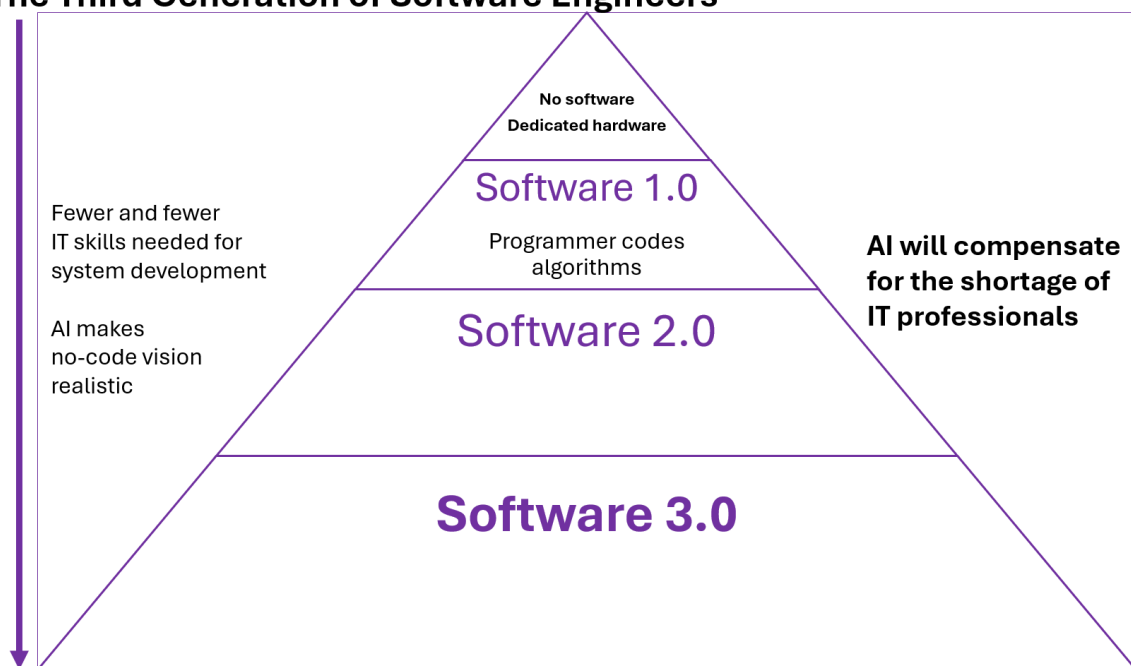


Figure 4.1: Towards "Software 3.0"

Recent academic and industry research demonstrates that integrating AI across all stages of the software development life cycle (SDLC)—from ideation to debugging—yields significant improvements in development speed, code quality, and team efficiency. One common cross-cutting task is summarizing information, which ranked second on the DORA survey's list of the top twelve AI-supported developer activities (DeBellis et al. 2024).

Below we present key insights supported by empirical studies and real-world deployments.



Holistic Involvement

Incorporating AI tools from the earliest phases of a software project enables not just code generation, but proactive engagement in architectural planning, requirements refinement, and system design. Hymel (2024) proposes the “AI-native” SDLC methodology, emphasizing a shift in developer roles from active creators to validators and orchestrators. In this model, AI can propose high-level strategies, identify design anti-patterns, and assist in risk modeling early on.

Tiered Model Utilization

While general-purpose models such as GPT-4 are capable of handling a wide variety of tasks including summarization, refactoring, and low-complexity implementations, highly specialized tasks benefit from using higher-reasoning models like OpenAI’s O1. These advanced models demonstrate superior performance in edge-case handling, domain-specific logic, and recursive reasoning. According to the study presented in Sajja et al. (2024), tiered model orchestration can reduce post-review error rates by up to 23% in enterprise applications.

- *GPT-4o: Mastering Human Communication*

GPT-4o excels in understanding the nuances of human communication. It adapts effortlessly to natural language, enabling frictionless and highly effective collaboration. This makes it an exceptional partner for creative tasks, brainstorming, and iterative refinement where fluid interaction and contextual awareness are critical.

- *O1: Precision in Complex Reasoning*

The O1 model, on the other hand, shines in tasks requiring structured commands and precise reasoning. It is particularly adept at tackling intricate logic, domain-specific challenges, and reasoning-intensive problems. While it may require more deliberate guidance, its ability to address complex scenarios makes it invaluable for highly technical tasks. This distinction highlights the synergy between models like GPT-4o and O1. By strategically combining their strengths—GPT-4o for seamless collaboration and O1 for deep reasoning—teams can achieve remarkable outcomes across the software life cycle.

GPT-5 (OpenAI 2025) is an LLM that combines a smart, fast model for answering most questions with a deeper reasoning model for harder problems. An integrated, real-time router quickly determines which model to use based on the type of conversation, its complexity, the necessary tools, and the user’s explicit intent (e.g., “Think hard about this” as part of the prompt).





4.2 Human-AI Interaction in Software Development

This section provides an overview of the preliminary findings regarding the new type of collaboration between humans and AI in software development.

4.2.1 Impact of AI Coding Assistants on Productivity and Efficiency in Development Workflows

The integration of large language model (LLM)–powered coding assistants into software development workflows is demonstrating notable impacts on developer productivity and efficiency. In a controlled experiment, developers using GitHub Copilot—an AI pair programmer based on OpenAI’s Codex model—completed a programming task 55.8% faster on average than those without it (Kalliamvakou 2022). In the same study, 78% of the AI-assisted group completed the task successfully versus 70% in the control group, with task time reduced from 2.7 to 1.2 hours.

Amazon reported similar gains with AWS’s CodeWhisperer: in early trials, developers completed tasks 57% faster and were 27% more likely to succeed (Sivasubramanian 2023). These tools reduce the time developer spend on boilerplate code, enabling them to focus on complex and creative challenges. Moreover, GitHub’s 2023 survey of over 2,000 developers found that 88% felt more productive with Copilot, 87% said it conserved mental energy, and many reported more time “in flow” and less frustration (Kalliamvakou 2022).

Taken together, both academic and industry findings suggest that LLM-based coding assistants can significantly enhance software development workflows, increasing both speed and developer satisfaction.

4.2.2 Collaboration Patterns Between Human Developers and AI Systems

As AI tools become more deeply embedded in development workflows, researchers are exploring the new patterns of collaboration between developers and LLMs. Human–AI “pAIr” programming is often compared to traditional pair programming. A recent systematic review by Ma, T. Wu, and Koedinger (2023) found that while human–AI pairs offer benefits similar to human-only pairs, their success depends on how well the AI complements the developer’s skills.

Developers tend to use AI assistants for tasks such as generating code snippets, completing functions, or explaining existing code, while retaining human oversight for reviewing, testing, and refining. Treude and Gerosa (2025) introduce a taxonomy of 11 interaction modes in AI–developer collaboration, including auto-complete, command-driven code generation, and conversational assistance. This taxonomy emphasizes that developers fluidly switch between accepting, editing, or rejecting AI suggestions based on context.

In practice, teams often treat AI assistants as “junior developers”—useful for drafting and prototyping, but not yet ready for autonomous decision-making. As noted in the survey by Stack Overflow (2024), developers appreciate AI tools for quick answers to scoped questions and boilerplate generation, but rely on human judgment for architecture and domain-specific reasoning.



4.2.3 Organizational and Process-Level Adaptations for AI Integration

Integrating generative AI into software development also requires organizational and process-level adaptations to improve human-AI interaction. To fully leverage the benefits of AI in software engineering, many organizations are restructuring their workflows, tools, and team practices. A key emerging strategy involves contextualizing AI coding assistants with project-specific knowledge. For example, Pinto et al. (2024) describe a case in which a company deployed a customized LLM assistant, “StackSpot AI,” which integrated proprietary API documentation and domain-specific content. In trials with 62 developers, this contextualized tool produced more accurate outputs and significantly improved efficiency compared to generic models.

However, incorporating custom knowledge into AI systems also introduces new challenges. Teams have to manage dynamic context updates and address inconsistencies in AI outputs when handling complex codebases. Beyond tool customization, companies are establishing internal policies for AI use—defining when to trust AI-generated code, how to structure code reviews, and how to train developers in effective prompting techniques.

A McKinsey survey of generative AI adoption across industries (Singla et al. 2025) found that, overall, 18% of respondents use it for software engineering. The technology sector, however, reached 36%. Using AI tools at scale can enable faster delivery and improved code quality. These benefits may encourage broader organizational changes, such as shorter iteration cycles and increased automation in CI/CD pipelines. Nevertheless, experts emphasize that AI’s effectiveness varies by task complexity and that human oversight remains essential for quality control and system-level coherence.

In everyday workflows, AI is now often embedded in IDEs, version control systems, and collaboration tools. Development processes are evolving to accommodate “AI-in-the-loop” practices—such as drafting code or tests with AI and assigning humans to verify, refine, and merge outputs. Over time, teams are learning how to strategically divide work between machines and humans, using AI for repetitive tasks and reserving critical thinking and architecture for human experts (JetBrains 2023).

4.2.4 Human Oversight, Prompting Strategies, and Feedback Loops with AI

Effective use of AI in software development necessitates strong human oversight. AI-generated code can be syntactically correct yet semantically flawed, insecure, or poorly adapted to the specific project context (Jiang et al. 2025). Developers must remain “in the loop,” reviewing, testing, and refining outputs to ensure safety and correctness.

One major concern is over-reliance: when developers accept AI suggestions without understanding them, it may lead to reduced skill development or increased vulnerabilities (Ma, T. Wu, and Koedinger 2023). Therefore, iterative feedback loops are essential. Developers must prompt clearly, review critically, and test rigorously. Jiang et al. (2025) emphasize the impact of prompt phrasing on model performance and recommend structured prompting, step-by-step reasoning, and constraint-based instructions to improve reliability.

Best practices now include decomposing problems into subtasks, instructing the AI explicitly, and expecting the AI to ask clarifying questions—features not yet robustly implemented in most current tools. Developers often spend significant time orchestrating these interactions: clarifying prompts, running test cases, and correcting hallucinated logic. However, these efforts help build a reliable feedback loop where human decisions govern machine suggestions (Jiang et al. 2025).



4.2.5 Productivity, Efficiency, Creativity, and Challenges in AI-Assisted Development

Early reports suggest that AI-human collaboration yields not only productivity and efficiency gains but also enhances developer satisfaction and creativity. Controlled experiments and industry surveys (e.g., GitHub Copilot, AWS CodeWhisperer) report coding speed increases of 50-100%, freeing developers from boiler-plate tasks and enabling deeper focus on complex or strategic work (Kalliamvakou 2022; Sivasubramanian 2023).

METR conducted a randomized controlled trial (Becker et al. 2025) to understand how early-2025 AI tools affect the productivity of experienced open-source developers working on their own repositories. Surprisingly, the study found that when developers use AI tools, they take 19% longer than without—AI makes them slower. The study authors view this result as a snapshot of early-2025 AI capabilities in one relevant setting; as these systems continue to rapidly evolve, METR plans on continuing to use this methodology to help estimate AI acceleration from AI R&D automation.

Surveys also indicate improved developer morale and less cognitive fatigue when AI assistants are used to automate repetitive work. A Stack Overflow pulse survey (Stack Overflow 2024) revealed that 27% of developers reinvest saved time into strategic thinking or refactoring, and many describe AI as a creative partner that helps them try new ideas quickly. This aligns with prior research that conceptualizes creativity in software engineering as clever reuse rather than pure novelty (Inman, D'Angelo, and Vasilescu 2024). Other studies emphasize that the impact of generative AI on creativity requires further systematic investigation (Jackson, Vasilescu, et al. 2025), while broader surveys highlight the role of creativity in shaping software engineering practices (Jackson, Hoek, et al. 2025). Recent work also points to new opportunities in AI-inspired design that extend beyond traditional coding tasks (Jialiang Wei et al. 2025). In particular, Mastropaolo, Escobar-Velásquez, and Linares-Vásquez (2025) examine the interplay between AI-driven automation and human innovation, noting that these two components can complement each other to preserve and even enhance intrinsic human creativity.

However, these benefits come with notable challenges. LLMs can introduce subtle bugs, incorrect suggestions, or security risks, particularly in unfamiliar domains. Trust and adoption also vary: developers often lose confidence after early mistakes (Ziegler et al. 2022; Ma, T. Wu, and Koedinger 2023). Inconsistent behavior between users, lack of transparency, and compliance concerns (e.g., licensing, explainability) further complicate integration (Kalliamvakou 2022; Jiang et al. 2025).

To address these challenges, organizations are adopting structured AI usage protocols, integrating human-in-the-loop quality assurance processes, and investing in training for effective prompting. The consensus is that AI's promise is best realized when human judgment complements automated code generation—transforming AI from a tool into a collaborative partner in development (Jiang et al. 2025).





4.2.6 Key Areas of Impact

Recent empirical studies and industry experiences have provided valuable insights into the practical implementation of LLMs in software development environments. These findings illuminate both the potential and challenges of AI integration in real-world development scenarios. Key Areas of impact include the aspects discussed below.

Developer Workflow Integration

Studies of the GitHub Copilot workspace (Dohmke 2024) demonstrate that AI-assisted development tools are most effective when seamlessly integrated into existing IDE environments. Key findings include:

- 96% reduction in time spent on boilerplate code generation
- Significant improvement in documentation quality
- Enhanced developer focus on high-level design decisions

Pair Programming Dynamics

Research on pair programming with generative AI (Spinellis 2024) reveals a shifting paradigm in collaborative development:

- AI tools serve as “always-available” programming partners
- Enhanced problem-solving through continuous feedback
- Reduced cognitive load for routine tasks
- Improved code review efficiency through AI-assisted analysis

Knowledge Transfer and Learning

The systematic literature review on LLMs for software engineering by X. Hou et al. (2024) highlights several educational benefits:

- Accelerated learning curve for new technologies
- Better understanding of complex codebases
- Improved code documentation comprehension
- Enhanced debugging skills through AI-assisted analysis

Development Process Optimization

Studies of generative AI in the software development life cycle identify several process improvements. As mentioned in Chapter 3, these improvements include a reduction in the time spent on initial code writing and documentation, as well as improvements in code review and correction workflows and enhanced test coverage and quality.



4.3 Case Study: Continuous LLM Engagement for Warehouse Routing Optimization

Within the context of our study, an experienced Accenture software engineer conducted an experiment with state-of-the-art generative AI, using a hypothetical though realistic problem description from current business practice. This example, which will be presented below, serves to illustrate that continuous LLM engagement across software development phases can create measurable value.

4.3.1 Example User Story

Jeremy Utley argues that the biggest unlock comes when teams treat AI as a thought partner woven through every creative step, rather than a one off tool (Utley 2025b; Utley 2025a). The following example user story, worked on by a single developer, illustrates how a continuous partnership can reduce research time, increase learning, and improve the quality of solutions throughout the entire software development life cycle.

User Story:

“As a logistics analyst, I need a route optimization algorithm that minimizes warehouse picking time so that we can reduce operational delays and improve efficiency.”

A conventional team would first spend hours comparing graph routing libraries, prototyping several options in turn. With an LLM engaged from the outset that research window shrinks to minutes, and the human developers acquire new knowledge hands on through dialog with the model. Generative AI stays in the loop for every phase below; the context it carries forward compounds, yielding measurable gains in speed, quality and developer mastery.





4.3.2 Experiment Environment

Since no single platform yet spans the full software life cycle, the evaluation was conducted inside the native OpenAI ChatGPT web UI—effectively a stand in for the integrated tooling we expect to emerge.

- **Workspace:** All work shared one context window. When token pressure surfaced, we invoked “Continue in new chat”, which auto summarizes the thread and seeds a fresh conversation, preserving continuity while resetting the token budget.
- **Model mix:**
 - **GPT 4o** (multimodal): Primary partner for conversational brainstorming: rapid ideation, voice enabled dialog, and image-based prompts; produces first pass scaffolds and keeps the discussion flowing in natural language.
 - **GPT 4.1** (≈1 M token context): Activated for instruction dense coding once artifacts grew beyond the effective window of 4o.
 - **o3 / o4 mini:** Selected for narrow, high precision reasoning tasks such as algorithmic fine tuning and cost function design.
- **Switch strategy:** The assistant periodically self-reported attention saturation. On confirmation we requested a concise roll up summary, then opened a new chat with that summary as the system prompt.

This orchestration demonstrates how future life-cycle suites could route work to specialized models behind the scenes while the user experiences a seamless, single threaded workflow.

4.3.3 Phase 1: Requirements Analysis

This phase was conducted using GPT-4o, OpenAI’s multimodal model, to simulate the early collaboration between a human analyst and an AI assistant. The model supported conversational, voice-based brainstorming as well as visual input where needed (e.g., warehouse diagrams or screen captures). This enabled rapid iteration and intuitive exploration of constraints, KPIs, and business objectives.

- LLM parses stakeholder notes, historical incidents and industry standards to draft precise, testable requirements.
- Surfaces key business drivers (e.g., pick time KPIs, labor cost, scalability).
- Highlights additional goals such as cost reduction and overall operational scalability, ensuring the algorithm aligns with broader warehouse performance objectives.
- Enumerates constraints and success metrics (directional aisles, dynamic obstacles, priority zones).

Deliverables: Requirements spec • Prioritized goals & constraints • Measurable KPIs

Example prompt (GPT-4o):

- *“Based on this user story and a sample warehouse layout diagram, help me define measurable requirements including route constraints and KPIs like average picking time.”*



4.3.4 Phase 2: Technology Evaluation & Refinement

This phase used GPT-4o, chosen for its ability to conduct interactive technology evaluations. The model provided instant access to performance characteristics, code examples, and trade-off comparisons across multiple libraries. Image prompts and shared architecture diagrams were used where helpful to clarify routing topology and constraints. Additionally, when deeper comparative analysis or architectural trade-offs were required, the assistant leveraged its deep research capabilities—synthesizing open-source documentation, academic papers, and community insights to provide well-reasoned, context-aware technology evaluations. This greatly exceeded the typical summarization of library features, offering strategic input grounded in real-world applicability.

- Humans would normally study multiple candidate libraries end to end, but the LLM performs that sweep in minutes and produces a data driven shortlist.
- Benchmarks NetworkX, SciPy csgraph, and Google OR Tools for performance, complexity, scalability and maintainability.
- The LLM cites community feedback and production case studies.

Deliverables: Curated library shortlist • Trade off matrix • Decision rationale document

Example *deep research* prompts (GPT-4o):

- *“Act as a senior solutions architect. Provide a detailed comparison of NetworkX, OR-Tools, and Scipy.csgraph for large-scale warehouse routing. Include performance benchmarks, production use cases, active maintenance history, and long-term community support.”*
- *“Compare NetworkX, OR-Tools, and Scipy.csgraph for implementing warehouse route optimization. Prioritize maintainability and support for directional edges.”*

4.3.5 Phase 3: Rapid Prototyping

This phase involved dynamic model selection depending on complexity:

- GPT-4o for conversational prototyping, quick iteration, and multimodal input (e.g., visual problem statements or sketches).
- GPT-4.1 was used for extended code sessions requiring deeper architectural context or multiple interdependent components.
- o3 / o4-mini were selected for narrow tasks involving algorithm fine-tuning, heuristic design, and precise test validation workflows.
- AI generates scaffold code, unit test stubs and inline docs for the chosen library.
- Suggests enhancements (one-way paths, capacity constraints, live re-routing).
- Produces detailed explanations and justification for each code decision, boosting maintainability and onboarding speed.
- Developers learn new APIs far faster than by passive documentation reading, because the LLM acts as an on-demand tutor and pair programmer.

The example of the LLM generated scaffold (using Python and NetworkX) is shown in Figure 4.1. Single prompt used:



```
1 import networkx as nx
2 from dataclasses import dataclass
3 from typing import List, Tuple
4
5 @dataclass
6 class WarehouseEdge:
7     src: str
8     dst: str
9     travel_time: int      # seconds
10    capacity: int = 1      # max pallets that can be moved simultaneously
11    blocked: bool = False # dynamic obstacle flag
12
13 # ---- Build directed graph with capacities & dynamic obstacles ----
14 edges: List[WarehouseEdge] = [
15     WarehouseEdge("A", "B", 4, capacity=2),
16     WarehouseEdge("A", "C", 2, capacity=1),
17     WarehouseEdge("C", "D", 3, capacity=1),
18     WarehouseEdge("B", "D", 10, capacity=3),
19     WarehouseEdge("B", "C", 5, capacity=1),
20 ]
21
22 G = nx.DiGraph()
23 for e in edges:
24     if not e.blocked: # filter out temporary obstacles
25         G.add_edge(e.src, e.dst, weight=e.travel_time, capacity=e.capacity)
26
27 # ---- Heuristic: penalise low capacity edges during peak load ----
28 def cost(u, v, data):
29     base = data["weight"]
30     cap_penalty = 5 if data["capacity"] == 1 else 0
31     return base + cap_penalty
32
33 start, end = "A", "D"
34 path = nx.shortest_path(G, start, end, weight=cost)
35 print("Optimised route (capacity aware):", " -> ".join(path))
36
```

Figure 4.1: AI-generated Python code

- “Generate a prototype that finds the fastest path while preferring high-capacity aisles and ignoring blocked routes.”

The example code was generated in under 30 seconds from the single prompt: “Create a minimal prototype that finds the fastest picking path between two warehouse nodes.”

Deliverables: Running prototype • Commented codebase • Design decision log

Example prompts:

- GPT-4o: “Sketch the first iteration of this pathfinding prototype using NetworkX—assume capacity constraints and avoid blocked edges.”
- GPT-4.1: “Now expand the prototype to support concurrent path optimization for multiple pickers.”
- o3/o4-mini: “Design a heuristic that penalizes low-capacity edges more aggressively during peak load.”



4.3.6 Phase 4: Validation, Testing & Optimization

GPT-4o was used for generating test cases, analyzing logs, and providing multimodal debugging assistance. Screen captures and command-line output were interpreted directly by the model, speeding up troubleshooting and refinement workflows.

- LLM auto creates exhaustive test suites, synthetic data and edge case scenarios using both traditional condition-based and property-based testing strategies.
- The model generates synthetic input datasets tailored to stress-test boundary conditions and logic violations.
- The model executes regression tests and provides explanatory diffs that highlight the exact nature and location of any behavioral deviation.
- Multi-modal analysis (screen captures, CLI logs) accelerates debugging.

Deliverables: Test automation pipeline • Performance report & optimization checklist • Resolved edge case catalog

Example prompts (GPT-4o):

- **Property-based testing:** “Generate property-based tests that assert consistency between alternative paths in the warehouse graph and validate against known constraints.”
- **Synthetic edge cases:** “Create synthetic routing graphs that expose flaws in the pathfinding logic— include cycles, disconnected nodes, and variable-capacity traps.”
- **Regression diff:** “Summarize behavioral differences between two versions of the routing algorithm using sample input-output pairs.” and “Generate edge case test data for the warehouse routing graph—include cycles, dead ends, and variable capacities.”

4.3.7 Phase 5: Documentation & Knowledge Capture

Documentation was collaboratively generated using GPT-4.1 for its large-context capacity and structured format support. The model translated implementation logs and source code into user-facing documents, API references, and traceable diagrams.

- AI generates API docs, user manuals and architecture diagrams directly from code and design logs.
- Maintains a live knowledge graph linking requirements → code → tests → deployment artefacts for traceability.

Deliverables: Comprehensive doc set • AI maintained knowledge graph • Real time doc updates via CI hooks

Example prompt (GPT-4.1):

- “Create user-facing documentation and an architecture diagram based on this source code and prior design decisions.”



4.3.8 Phase 6: Deployment & Continuous Improvement

This final phase used a combination of two LLMs. GPT-4.1 handled deployment artefact summarization and automation scripts, while GPT-4o managed user-facing support flows and interactive monitoring logic.

- Post deployment, LLM plugs into monitoring frameworks to surface anomalies and recommend fixes before SLAs are breached.
- Voice-enabled interface offers zero touch Level 1 support; the model references the knowledge graph to resolve recurrent issues.
- Continuous analytics drive incremental algorithm tuning as warehouse conditions evolve.

Deliverables: AI augmented Level 1 to Level 3 support • Conversational ops interface • Real time optimization recommendations

Example prompts:

- GPT-4.1: “Summarize all code changes and CI configs introduced post-release.”
- GPT-4o: “Design a conversational L1 support flow that helps warehouse staff diagnose routing delays.”

4.3.9 Current Landscape & Future Potential

Most commercial AI tools (e.g., GitHub Copilot, Cursor) tackle only one SDLC slice—coding or debugging. That siloed approach captures a fraction of AI’s potential. Full life cycle engagement, as demonstrated above, unlocks far larger efficiency and quality gains. Integrated platforms that weave AI through every phase are emerging, but real success depends on lightweight, context aware integration rather than monolithic solutions. Echoing Utley’s call for an “AI first culture,” inaction is now the greater risk; organizations that reward experimentation will outpace those that don’t. In March 2025, Anthropic’s CEO made an executive forecast claiming that AI could write ≈90% of code within months (Tan 2025). Whether or not that timeline sticks, the sustainable corporate path is the one shown here: continuous, collaborative AI partnering with humans from idea to incident response.



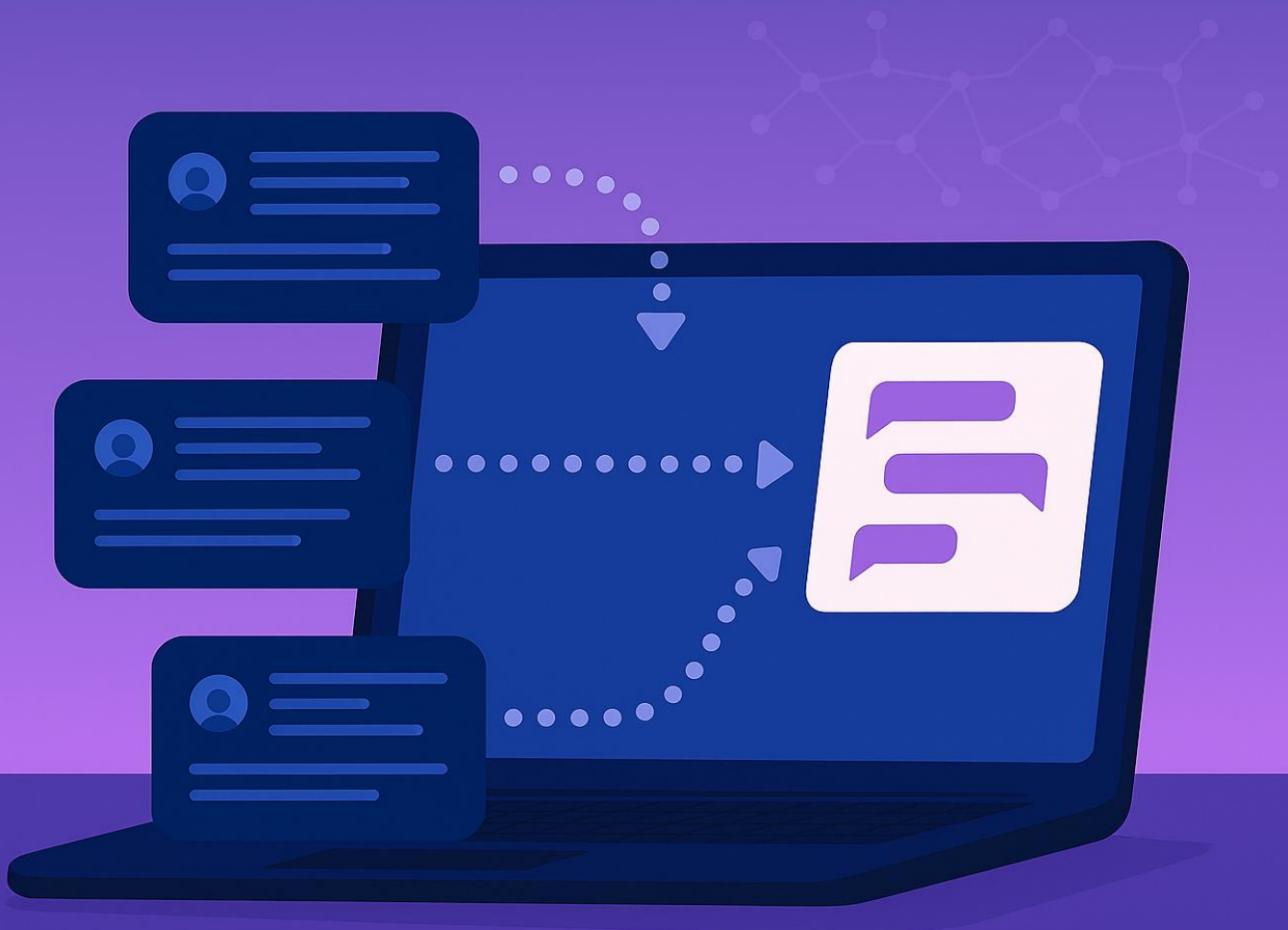


4.3.10 Practical Mitigation of Context Window Limits

Even the largest contemporary LLMs have finite context windows; very long sessions can trigger quality degradation or forgotten details. In practice the workaround is straightforward:

- **Periodic roll ups:** Ask the model to summarize the working context into a concise hand over brief, then start a fresh chat with that summary as the system prompt.
- **Thread sharding:** Maintain multiple parallel chats, each focused on a specific subtopic; this keeps token budgets manageable while preserving depth.
- **Self-reflection triggers:** Some practitioners simply ask, “is your attention window saturated?” and receive model reported signal to noise cues before deciding to roll over.

OpenAI’s new in thread summarization feature (available via the Assistants API and rolling out in ChatGPT’s “Continue in new chat” button) automates the hand-over step, making window management largely invisible to end users. Used together, these hygiene practices keep the compounding-context cycle alive well beyond any single chat session—and they cost far less cognitive overhead than re-reading logs manually.





4.3.11 Key Take Away

Engaging an LLM at every SDLC phase multiplies—not merely adds to—project agility, risk reduction and solution quality. The warehouse routing case illustrates how context retention and iterative human AI collaboration deliver benefits far beyond the narrow “AI just for coding” narrative (see Table 4.1). Working with LLMs across phases not only accelerated delivery but transformed the development process into a more exploratory and dialog-driven experience. Developers reported increased clarity, faster on-boarding to unfamiliar libraries, and reduced context-switching overhead. Echoing Jeremy Uteley’s view that “AI unlocks human potential when treated as a collaborator,” this approach empowered the human team to focus on high-level reasoning, trust-building, and creative flow—while the assistant handled complexity, search, and synthesis at scale.

SDLC phase	What a continuously involved AI adds	Why siloed “AI only at coding” misses the boat
Ideation & Backlog Grooming	Rapid brainstorming, competitive scans, AI drafted user stories & acceptance criteria	Later AI can’t infer hidden rationale—traceability lost, code suggestions drift
Requirements Engineering	Ambiguity checks, glossary alignment, auto generated testable requirements	Vagueness persists; downstream code hallucinates around missing detail
Architecture & Design	Pattern suggestions, library trade off matrices, early security flags	Wrong architectural choice becomes expensive to undo
Implementation	Pair programming, boilerplate generation, instant refactor advice	Captures only incremental coding speed up, ignores upstream waste
Test / QA	Unit test scaffolds, fuzz/property tests, coverage gap highlights	Manual test creation lags behind code, defects leak forward
Deployment & Ops	Auto generated CI/CD, log anomaly summaries	/

Table 4.1: Compounding context effect of continuous LLM engagement





4.4 Adoption of Generative AI into Software Development

Evaluation studies of generative AI-assisted software design highlight the importance of seamless integration with existing development processes (Fischer and Lanquillon 2024). Further adoption studies emphasize the need for standardized practices and clear operational models. For example, Sauvola et al. (2024) distinguish four adoption scenarios ranging from traditional development operations to human-in-the-loop models, providing a framework for positioning GenAI integration. Trust also emerges as a critical factor: Choudhuri et al. (2025) identify quality, functional value, and goal maintenance of GenAI systems as key determinants of developer trust, which in turn affects intentions to adopt these tools. Other surveys highlight practical challenges around developer experience and documentation needs (Tenbergen and Krusche 2024; Russo 2024).

Key findings across these studies include:

- **Need for standardized practices in AI tool usage:** Clear adoption frameworks help teams position and scale GenAI tools effectively (Sauvola et al. 2024).
- **Importance of maintaining code consistency:** Developers stress the need to adapt coding standards to incorporate AI-generated contributions (Tenbergen and Krusche 2024).
- **Requirements for documentation and traceability:** Documenting AI-assisted contributions is essential for long-term maintainability and auditing (Russo 2024).
- **Trust and inclusivity:** Adoption depends on developer trust in tool quality and inclusivity, shaped by cognitive styles and perceived value (Choudhuri et al. 2025).

4.4.1 Challenges and Considerations

Successful adoption of generative AI requires careful consideration of the challenges at the team level and an appropriate cost-benefit analysis.

Team Adaptation

Studies of practical implementations reveal several key factors for successful team adoption:

- **Clear guidelines on AI tool usage:** Establishing well-defined policies ensures consistent and effective use of AI tools across teams (Codacy 2023).
- **Maintaining coding standards:** Adapting existing coding standards to accommodate AI-generated code helps maintain code quality and readability (Codacy 2023).
- **Balancing AI assistance with human oversight:** While AI can automate routine tasks, human review remains essential to catch nuanced issues and ensure correctness (Codacy 2023).
- **Training for effective AI tool utilization:** Providing training programs enhances developers' ability to leverage AI tools effectively, leading to increased adoption, productivity, and efficiency (Pearson 2025).



Cost-Benefit Considerations

Evaluating the return on investment (ROI) of AI adoption involves analyzing various factors (Khan 2025):

- **Initial setup and training costs:** Upfront investments include purchasing AI tools and training staff, which can be substantial.
- **Productivity and efficiency gains over time:** AI tools can lead to significant productivity and efficiency improvements, offsetting initial costs.
- **Resource allocation optimization:** Automating routine tasks allows teams to focus on higher-value activities, optimizing resource utilization.
- **Long-term maintenance implications:** Ongoing maintenance and updates of AI tools require continuous investment and planning.

4.4.2 Recommendations for Implementation

Successful AI adoption requires a phased integration of AI into software development processes and adequate preparation of the development team. Panyam and Gujar (2025) examined how the increasing use of AI agents is changing the field of software engineering. Since the rise of AI presents both opportunities and challenges, the authors emphasize the crucial role of engineering leaders in guiding their teams through this transformation. This requires leaders to adopt new strategies to maximize productivity while managing increased complexity.

Phased Integration

A gradual approach to integrating AI into software development processes ensures manageable adoption and minimizes disruption (AWS 2025). For agent-based frameworks, phased adoption is particularly important. Organizations can begin with single agent use cases (e.g., automating unit test generation or code review) and then expand toward multi-agent workflows that involve coordination between specialized roles. Because multi-agent systems may display emergent behaviors, phased integration helps identify potential risks and establish oversight mechanisms early.

- **Start with low-risk, well-defined tasks:** Introduce agents into tasks with clear parameters and limited scope to build confidence and demonstrate value.
- **Gradually expand to multi-agent workflows:** Extend adoption toward more complex applications where agents collaborate or assume specialized roles (e.g., planner, executor, critic).
- **Establish clear metrics for success:** Define measurable goals that account for both productivity and quality, while monitoring agent interactions and coordination overhead.
- **Regular evaluation and adjustment:** Continuously monitor AI integration outcomes, especially in multi-agent settings, and refine processes to align with evolving objectives.



Team Preparation

Preparing the development team is crucial for successful AI adoption (Team Ascend 2025). Agentic systems require developers not only to learn how to use AI tools but also how to supervise and coordinate multiple interacting agents. Training should therefore emphasize role allocation, clarify boundaries of agent autonomy, and establish review processes that ensure human oversight remains central.

- **Comprehensive training programs:** Equip team members with the skills to supervise agent workflows, including orchestration and troubleshooting of multi-agent interactions.
- **Clear guidelines for agent tool usage:** Define appropriate use cases, boundaries of autonomy, and escalation points for human review.
- **Established review processes:** Implement governance mechanisms to oversee outputs from single and multi-agent systems, ensuring accountability and code quality.
- **Regular feedback collection and analysis:** Collect ongoing feedback on agent behavior and integration challenges to inform iterative improvements and policy adjustments.

4.4.3 Future Considerations

Organizations should anticipate and prepare for emerging trends (AWS 2025):

- **Evolution of AI capabilities and tools:** Stay informed about advancements in AI to leverage new opportunities.
- **Changes in developer skill requirements:** Adapt hiring and training strategies to meet the evolving skill demands prompted by AI integration.
- **New security and privacy challenges:** Proactively address potential risks associated with AI, including data privacy and ethical considerations.
- **Emerging best practices and standards:** Contribute to and adopt industry standards to guide responsible AI use.

The capabilities of current generative AI suggest a fundamental reshaping of development team workflows, emphasizing not just automation but augmentation, context-awareness, and human-aligned decision support. Our synthesis of recent research provides a framework for organizations considering or implementing AI-assisted development practices. Emphasizing practical experiences and empirical evidence offers valuable guidance for navigating the challenges and opportunities presented by these new technologies.



5. Conclusion





This final section of our study summarizes the findings and presents a prospective view on future progress in the use of generative AI for professional software engineering. Accenture technology leaders Daugherty and Wilson, who described the concept of new types of hybrid human and machine roles already in 2018, have updated their seminal work to include recent developments in generative AI (Daugherty and Wilson 2024). The authors favor the notion of human and machine, instead of human vs. machine. Figure 5.1 illustrates the key idea of the “missing middle”, where intelligent machines are amplifying our skills and collaborating with us to achieve productivity and efficiency gains that have previously been impossible.

<i>Lead</i> <i>Empathize</i> <i>Create</i> <i>Judge</i>	<i>Train</i> <i>Explain</i> <i>Sustain</i>	<i>Amplify</i> <i>Interact</i> <i>Embody</i>	<i>Transact</i> <i>Iterate</i> <i>Predict</i> <i>Adapt</i>
H	Humans complement machines	AI gives humans superpowers	M
	+		
<i>Human-only activity</i>	Human and machine hybrid activities		<i>Machine-only activity</i>

Figure 5.1: Humans and machines collaborate to fill the missing middle; from Daugherty and Wilson (2024)

5.1 Toward AI-Enhanced Software Development

Leveraging generative AI throughout each software development phase significantly enhances human-AI collaboration, boosting agility, minimizing risks, and elevating overall solution quality. Generative AI goes beyond just coding, it helps with problem definition, library documentation research, and decision-making.

Initially, many AI-based tools, such as GitHub Copilot, Cursor, and similar products, predominantly focused on singular phases of software development, typically coding or debugging. While these tools significantly increased productivity in their respective areas, their isolated approach limited the full potential of AI in software creation. LLMs can unlock substantially greater efficiency gains and innovation by being actively engaged and contextually aware throughout every phase of development, from initial requirements gathering through continuous deployment and support.

Emerging frameworks and integrated AI platforms that cover the entire software life cycle are gradually appearing in the market. A critical shift towards end-to-end life-cycle support is indicated by these comprehensive solutions, even though they sometimes risk being overly complex. The ideal solution would balance AI-driven efficiency with streamlined, lightweight integrations, ensuring AI’s capabilities are optimally utilized without unnecessary overhead.



5.2 Outlook

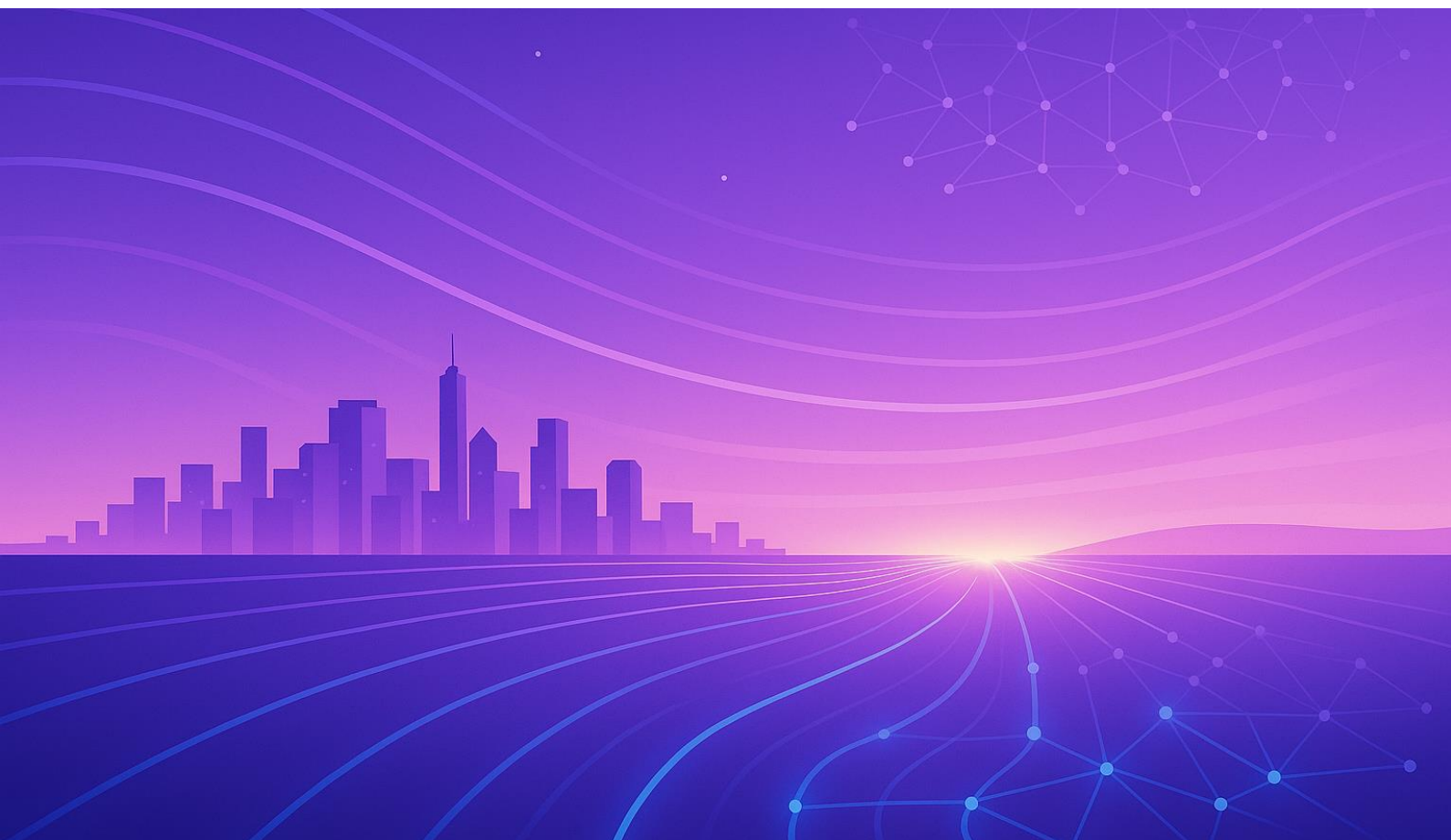
In March 2025, the CEO of Anthropic predicted that within a few months, AI could generate approximately 90% of all software code, with the potential to approach 100% automation within a year or so (CFR 2025).

While this bold prediction highlights AI's transformative potential in software development, a measured, integrated approach—engaging AI comprehensively across the entire software life cycle—appears more realistic and sustainable, particularly within corporate environments. According to Ahmed et al. (2025), AI will seamlessly integrate into software engineering. AI will accomplish tedious, repetitive, effort-intensive, error-prone activities, whereas developers will focus on creative problem-solving, innovation (Ahmed et al. 2025), supervision and validation (Lyu et al. 2025).

Future research will face the following challenges (Ahmed et al. 2025): (1) how AI will interact with developers, (2) responsible AI (e.g., bias, intellectual property of code, data privacy, authorized usage of data, security, transparent training of models, explainability of AI-generated suggestions, and liability for failures of AI-generated code (Lyu et al. 2025)), and (3) availability of large, labeled datasets of industrial-scale applications for benchmarking and potentially training.

Cox et al. (2024) conclude:

“Staying adaptable and open to new tools will be the key to staying ahead because, as we all know, today is the worst AI will ever be.”





6. Acknowledgements



We would like to thank Daniel Klan, Lorenzo Toscano, Felix Klemm, Lucian Melnic, Philipp Neuheisel at Accenture and Daniel Porta, Sönke Knoch, Tilman Becker, Ludger van Elst, Norbert Reithinger at DFKI for their valuable advice during the creation of this white paper, as well as for their thorough review of the draft version.





7. References



Abendroth-Dias, Kulani et al. (June 2025). Generative AI Outlook Report: Exploring the Intersection of Technology, Society and Policy. Technical Report JRC142598. Luxembourg: European Commission Joint Research Centre (JRC). URL: <https://doi.org/10.2760/1109679> (visited on 08/15/2025).

Abrahamsson, Pekka et al. (2024). “ChatGPT as a Fullstack Web Developer – Early Results”. In: Agile Processes in Software Engineering and Extreme Programming – Workshops. Ed. by Philippe Kruchten and Peggy Gregory. Lecture Notes in Business Information Processing 489. Cham: Springer, pp. 201–209. URL: https://doi.org/10.1007/978-3-031-48550-3_20 (visited on 08/15/2025).

Abramatic, Jean-François, Roberto Di Cosmo, and Stefano Zacchioli (Oct. 2018). “Building the Universal Archive of Source Code”. In: Communications of the ACM 61.10, pp. 29–31. DOI: 10.1145/3183558 URL: <https://cacm.acm.org/opinion/building-the-universal-archive-of-source-code/> (visited on 08/15/2025).

Agarwal, Shivali et al. (2024). “Tutorial Report on Legacy Software Modernization: A Journey From Non-AI to Generative AI Approaches”. In: Proceedings of the 17th Innovations in Software Engineering Conference (ISEC ’24). ACM, 19:1–19:3. URL: 10.1145/3641399.3641434 (visited on 08/15/2025).

Ahmed, Iftekhar et al. (2025). “Artificial Intelligence for Software Engineering: The Journey So Far and the Road Ahead”. In: ACM Transactions on Software Engineering and Methodology 34.5, 119:1–119:27. URL: <https://doi.org/10.1145/3719006> (visited on 08/15/2025).

Aleti, Aldeida (2023). Software Testing of Generative AI Systems: Challenges and Opportunities. Computing Research Repository (CoRR). URL: <https://doi.org/10.48550/arXiv.2309.03554> (visited on 08/15/2025).

Allspaw, John and Paul Hammond (2009). 10 Deploys per Day: Dev & Ops Cooperation at Flickr. O’Reilly Velocity 2009, conference presentation. URL: <https://paulhammond.org/2009/06/velocity/10deploys.pdf> (visited on 08/15/2025).

Amaro, Ricardo, Rúben Pereira, and Miguel Mira da Silva (2024). “DevOps Metrics and KPIs: A Multivocal Literature Review”. In: ACM Computing Surveys 56.9, 231:1–231:41. URL: <https://doi.org/10.1145/3652508> (visited on 08/15/2025).

Ananthaswamy, Anil (Mar. 9, 2023). “In AI, is Bigger Always Better?” In: Nature 615.7951, pp. 202–205. URL: <https://doi.org/10.1038/d41586-023-00641-w> (visited on 08/15/2025).

Anthropic (Nov. 25, 2024). Introducing the Model Context Protocol. Anthropic. URL: <https://www.anthropic.com/news/model-context-protocol> (visited on 08/15/2025).

Arora, Chetan, John Grundy, and Mohamed Abdelrazek (2023). Advancing Requirements Engineering through Generative AI: Assessing the Role of LLMs. Computing Research Repository (CoRR). URL: <https://doi.org/10.48550/arXiv.2310.13976> (visited on 08/15/2025).

Assunção, Wesley K. G. et al. (2025). “Contemporary Software Modernization: Strategies, Driving Forces, and Research Opportunities”. In: ACM Transactions on Software Engineering and Methodology 34.5, 142:1–142:35. URL: <https://doi.org/10.1145/3708527> (visited on 08/15/2025).



AWS (Apr. 18, 2025). Best Practices for Using Generative AI in Software Development. Amazon Web Services. URL: <https://docs.aws.amazon.com/prescriptive-guidance/latest/strategy-accelerate-software-dev-lifecycle-gen-ai/best-practices.html> (visited on 08/15/2025).

Barr, Jeff (June 23, 2022). Now in Preview – Amazon CodeWhisperer – ML-Powered Coding Companion. AWS News Blog. URL: <https://aws.amazon.com/blogs/aws/now-in-preview-amazon-codewhisperer-ml-powered-coding-companion/> (visited on 08/15/2025).

Beck, Kent et al. (2001). Manifesto for Agile Software Development. Agile Alliance. URL: <https://agilemanifesto.org/> (visited on 08/15/2025). **Becker, Joel et al. (2025).** Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. Computing Research Repository (CoRR). URL: <https://doi.org/10.48550/arXiv.2507.09089> (visited on 08/15/2025).

Benaich, Nathan, Alex Chalmers, et al. (Oct. 10, 2024). State of AI Report 2024. Presentation. London, UK: Air Street Capital. URL: <https://www.stateof.ai/2024-report-launch> (visited on 08/15/2025).

Bender, Emily M. et al. (2021). “On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?” In: Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT ’21), pp. 610–623. URL: <https://doi.org/10.1145/3442188.3445922> (visited on 08/15/2025).

Bhatia, Shreya et al. (2024). “Unit Test Generation using Generative AI: A Comparative Performance Analysis of Autogeneration Tools”. In: Proceedings of the 1st International Workshop on Large Language Models for Code (LLM4Code ’24), pp. 54–61. URL: <https://doi.org/10.1145/3643795.3648396> (visited on 08/15/2025).

BlueOptima (June 13, 2023). DORA Metrics and SPACE Metrics: A Comparative Overview. BlueOptima Blog. URL: <https://www.blueoptima.com/dora-metrics-and-space-metrics-a-comparative-overview-for-software-development-leaders/> (visited on 08/15/2025).

Böhme, Marcel et al. (2025). “Software Security Analysis in 2030 and Beyond: A Research Roadmap”. In: ACM Transactions on Software Engineering and Methodology 34.5, 144:1–144:26. URL: <https://doi.org/10.1145/3708533> (visited on 08/15/2025).

Borg, Markus et al. (2023). “Human, What Must I Tell You?” In: IEEE Software 40.3, pp. 9–14. URL: <https://doi.org/10.1109/MS.2023.3244638> (visited on 08/15/2025).

Brundage, Miles et al. (2018). The Malicious Use of Artificial Intelligence: Forecasting, Prevention, and Mitigation. Technical Report. University of Oxford et al. DOI: 10.48550/arXiv.1802.07228. URL: <https://maliciousaireport.com/> (visited on 08/15/2025).

Burgueño, Lola et al. (2025). “Automation in Model-Driven Engineering: A Look Back, and Ahead”. In: ACM Transactions on Software Engineering and Methodology 34.5, 122:1–122:25. URL: <https://doi.org/10.1145/3712008> (visited on 08/15/2025).

Cámara, Javier et al. (2023). “On the Assessment of Generative AI in Modeling Tasks: An Experience Report with ChatGPT and UML”. In: Software and Systems Modeling 22.3, pp. 781–793. URL: <https://doi.org/10.1007/s10270-023-01105-5> (visited on 08/15/2025).

Carleton, Anita, Mark Klein, et al. (Nov. 2021). Architecting the Future of Software Engineering: A National Agenda for Software Engineering Research & Development. Study. Pittsburgh, PA: Carnegie Mellon University, Software Engineering Institute. URL: <https://insights.sei.cmu.edu/library/architecting-the-future-of-software-engineering-a-national-agenda-for-software-engineering-research-development/> (visited on 08/15/2025).

Carleton, Anita, Forrest Shull, and Erin Harper (2022). “Architecting the Future of Software Engineering”. In: Computer 55.9, pp. 89–93. URL: <https://doi.org/10.1109/MC.2022.3187912> (visited on 08/15/2025).



Castelvecchi, Davide (Aug. 7, 2025). “DeepMind and OpenAI Models Solve Maths Problems at Level of Top Students”. In: Nature 644.8075, p. 20. URL: <https://doi.org/10.1038/d41586-025-02343-x> (visited on 08/15/2025).

CFR (Mar. 10, 2025). CEO Speaker Series with Dario Amodei of Anthropic. Council on Foreign Relations (CFR). URL: <https://www.cfr.org/event/ceo-speaker-series-dario-amodei-anthropic> (visited on 08/15/2025).

Chacon, Scott and Ben Straub (2014). Pro Git. 2nd ed. Berkeley, CA: Apress. DOI: <https://doi.org/10.1007/978-1-4842-0076-6>. URL: <https://git-scm.com/book/en/v2> (visited on 08/15/2025).

Chen, Daihang et al. (2025). “LLM for Mobile: An Initial Roadmap”. In: ACM Transactions on Software Engineering and Methodology 34.5, 128:1–128:29. URL: <https://doi.org/10.1145/3708528> (visited on 08/15/2025).

Chen, Junkai et al. (2024). “Code Search is All You Need? Improving Code Suggestions with Code Search”. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE ’24), 73:1–73:13. URL: <https://doi.org/10.1145/3597503.3639085> (visited on 08/15/2025).

Chen, Mark et al. (2021). Evaluating Large Language Models Trained on Code. Computing Research Repository (CoRR). URL: <https://doi.org/10.48550/arXiv.2107.03374> (visited on 08/15/2025).

Chen, Zimin et al. (2021). “SequenceR: Sequence-to-Sequence Learning for End-to-End Program Repair”. In: IEEE Transactions on Software Engineering 47.9, pp. 1943–1959. DOI: <https://doi.org/10.48550/arXiv.1901.01808> URL: <https://doi.org/10.1109/TSE.2019.2940179> (visited on 08/15/2025).

Choudhuri, Rudrajit et al. (2025). “What Guides Our Choices? Modeling Developers’ Trust and Behavioral Intentions Towards GenAI”. In: Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE ’25), pp. 1691–1703. DOI: 10.48550/arXiv.2409.04099. URL: <https://doi.org/10.1109/ICSE55347.2025.00087> (visited on 08/15/2025).

Codacy (Dec. 22, 2023). Best Practices for Coding with AI in 2024. Codacy Blog. URL: <https://blog.codacy.com/best-practices-for-coding-with-ai> (visited on 08/15/2025).

Combemale, Benoit, Jeff Gray, and Bernhard Rumpe (2023). “ChatGPT in Software Modeling”. In: Software and Systems Modeling 22.3, pp. 777–779. URL: <https://doi.org/10.1007/s10270-023-01106-4> (visited on 08/15/2025).

Cox, Jason et al. (2024). “DevOps Enablement with GenAI: Augmenting Skills Through GenAI”. In: Enterprise Technology Leadership Journal 1.2, pp. 71–91. URL: <https://itrevolution.com/product/devops-enablement-with-genai/> (visited on 08/15/2025).

Daugherty, Paul R. and H. James Wilson (2024). Human + Machine: Reimagining Work in the Age of AI. 2nd ed. Boston, MA: Harvard Business Review Press.

Davila, Nicole, Jorge Melegati, and Igor Wiese (2024). “Tales From the Trenches: Expectations and Challenges from Practice for Code Review in the Generative AI Era”. In: IEEE Software 41.6, pp. 38–45. URL: <https://doi.org/10.1109/MS.2024.3428439> (visited on 08/15/2025).

Davis, Gwen (June 8, 2023). Developer Experience: What Is It and Why Should You Care? The GitHub Blog. URL: <https://github.blog/enterprise-software/collaboration/developer-experience-what-is-it-and-why-should-you-care/> (visited on 08/15/2025).

DeBellis, Derek et al. (2024). 2024 Accelerate State of DevOps. DORA Report 10. Google Cloud DevOps Research and Assessment (DORA) team. URL: <https://dora.dev/research/2024/dora-report/> (visited on 08/15/2025).



DeepSeek-AI et al. (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. Computing Research Repository (CoRR). URL: <https://doi.org/10.48550/arXiv.2501.12948> (visited on 08/15/2025).

DeSanto, David (June 22, 2023). Meet GitLab Duo, the Suite of AI Capabilities Powering Your Workflows. GitLab Blog. URL: <https://about.gitlab.com/blog/meet-gitlab-duo-the-suite-of-ai-capabilities/> (visited on 08/15/2025).

Di Cosmo, Roberto (2018). “Software Heritage: Collecting, Preserving, and Sharing All Our Source Code”. In: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE ’18), pp. 1–2. URL: <https://doi.org/10.1145/3238147.3241985> (visited on 08/15/2025).

Ding, Ao et al. (2024). “Generative AI for Software Security Analysis: Fundamentals, Applications, and Challenges”. In: IEEE Software 41.6, pp. 46–54. URL: <https://doi.org/10.1109/MS.2024.3416036> (visited on 08/15/2025).

Ding, Hao et al. (2024). “Reasoning and Planning with Large Language Models in Code Development”. In: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD ’24), pp. 6480–6490. URL: <https://doi.org/10.1145/3637528.3671452> (visited on 08/15/2025).

Dohmke, Thomas (June 21, 2022). GitHub Copilot Is Generally Available to All Developers. The GitHub Blog. URL: <https://github.blog/news-insights/product-news/github-copilot-is-generally-available-to-all-developers/> (visited on 08/15/2025).

— **(Apr. 29, 2024).** GitHub Copilot Workspace: Welcome to the Copilot-native Developer Environment. The GitHub Blog. URL: <https://github.blog/2024-04-29-github-copilot-workspace/> (visited on 08/15/2025).

“DOORS” (2017). “DOORS: A Tool to Manage Requirements”. In: Dick, Jeremy, Elizabeth Hull, and Ken Jackson. Requirements Engineering. Cham: Springer, pp. 187–206. URL: https://doi.org/10.1007/978-3-319-61073-3_8 (visited on 08/15/2025).

Duvall, Paul, Stephen M. Matyas, and Andrew Glover (2007). Continuous Integration. Improving Software Quality and Reducing Risk. Upper Saddle River, NJ: Addison-Wesley.

Ebert, Christof and David A. Gustafson (2024). “Software Measurement: Four Steps to Unlock Success”. In: Computer 57.12, pp. 104–109. URL: <https://doi.org/10.1109/MC.2024.3458368> (visited on 08/15/2025).

Ehtesham, Abul et al. (2025). A Survey of Agent Interoperability Protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP). Computing Research Repository (CoRR). URL: <https://doi.org/10.48550/arXiv.2505.02279> (visited on 08/15/2025).

Fan, Wenqi et al. (2024). “A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models”. In: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD ’24), pp. 6491–6501. DOI: 10.1145/3637528.3671470. URL: <https://advanced-recommender-systems.github.io/RAG-Meets-LLMs/> (visited on 08/15/2025).

Farquhar, Sebastian et al. (June 20, 2024). “Detecting Hallucinations in Large Language Models Using Semantic Entropy”. In: Nature 630.8017, pp. 625–630. URL: <https://doi.org/10.1038/s41586-024-07421-0> (visited on 08/15/2025).

Ferrando, Javier et al. (2024). A Primer on the Inner Workings of Transformer-based Language Models. Computing Research Repository (CoRR). URL: <https://doi.org/10.48550/arXiv.2405.00208> (visited on 08/15/2025).



Ferrara, Pietro, Vincenzo Arceri, and Agostino Cortesi (2024). “Challenges of Software Verification: The Past, the Present, the Future”. In: International Journal on Software Tools for Technology Transfer 26.4, pp. 421–430. URL: <https://doi.org/10.1007/s10009-024-00765-y> (visited on 08/15/2025).

Fettke, Peter and Chiara Di Francescomarino (2025). “Business Process Management and Artificial Intelligence”. In: KI – Künstliche Intelligenz 39.2. tex.ids: Flint.FettkeDiFrancescomarino25sn, pp. 67–79. URL: <https://doi.org/10.1007/s13218-025-00891-y> (visited on 08/15/2025).

Fettke, Peter and Wolfgang Reisig (2024). Understanding the Digital World: Modeling with HERAKLIT. tex.ids: Flint.FettkeReisig24. Cham: Springer. URL: <https://doi.org/10.1007/978-3-031-61898-7> (visited on 08/15/2025).

Fischer, Mahsa and Carsten Lanquillon (2024). “Evaluation of Generative AI-Assisted Software Design and Engineering: A User-Centered Approach”. In: Artificial Intelligence in HCI. Ed. by Helmut Degen and Stavroula Ntoa. Lecture Notes in Computer Science 14734. Cham: Springer, pp. 31–47. URL: https://doi.org/10.1007/978-3-031-60606-9_3 (visited on 08/15/2025).

Flexential (Oct. 8, 2024). Building the Future of AI: A Comprehensive Guide to AI Infrastructure. Flexential Blog. URL: <https://www.flexential.com/resources/blog/building-future-ai-infrastructure> (visited on 08/15/2025).

Forsgren, Nicole and Mik Kersten (2018). “DevOps Metrics”. In: Communications of the ACM 61.4, pp. 44–48. DOI: 10.1145/3159169. URL: <https://cacm.acm.org/practice/devops-metrics/> (visited on 08/15/2025).

Forsgren, Nicole, Margaret-Anne Storey, et al. (2021). “The SPACE of Developer Productivity: There’s More to It Than You Think”. In: Queue 19.1, pp. 20–48. URL: <https://doi.org/10.1145/3454122.3454124> (visited on 08/15/2025).

Fourney, Adam et al. (Nov. 2024). Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks. Technical Report MSR-TR-2024-47. Microsoft. DOI: 10.48550/arXiv.2411.04468. URL: <https://aka.ms/magentic-one-report> (visited on 08/15/2025).

Fowler, Martin (Jan. 18, 2024). Continuous Integration. MartinFowler.com. URL: <https://martinfowler.com/articles/continuousIntegration.html> (visited on 08/15/2025).

Franzen, Carl (Sept. 12, 2023). Salesforce Introduces New AI Assistant, Einstein Copilot, for All Its CRM Apps. VentureBeat. URL: <https://venturebeat.com/business/salesforce-introduces-new-ai-assistant-einstein-copilot-for-all-its-crm-apps/> (visited on 08/15/2025).

— **(Mar. 10, 2025).** What You Need to Know About Manus, the New AI Agentic System From China Hailed as a Second ‘DeepSeek Moment’. VentureBeat. URL: <https://venturebeat.com/ai/what-you-need-to-know-about-manus-the-new-ai-agentic-system-from-china-hailed-as-a-second-deepseek-moment/> (visited on 08/15/2025).

Full Scale (Apr. 16, 2025). AI Software Development: A Comprehensive Guide for Modern Businesses. Full Scale Blog. URL: <https://fullscale.io/blog/guide-to-successful-ai-software-development/> (visited on 08/15/2025).

Gao, Ya (May 13, 2024). Research: Quantifying GitHub Copilot’s Impact in the Enterprise with Accenture. The GitHub Blog. URL: <https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-in-the-enterprise-with-accenture/> (visited on 08/15/2025).

Gao, Yunfan et al. (2024). Retrieval-Augmented Generation for Large Language Models: A Survey. Computing Research Repository (CoRR). URL: <https://doi.org/10.48550/arXiv.2312.10997> (visited on 08/15/2025).



Gärtner, Alexander Elenga and Dietmar Göhlich (2024a). “Automated Requirement Contradiction Detection Through Formal Logic and LLMs”. In: Automated Software Engineering 31.2, p. 49. URL: <https://doi.org/10.1007/s10515-024-00452-x> (visited on 08/15/2025).

Gärtner, Alexander Elenga and Dietmar Göhlich (2024b). “Towards an Automatic Contradiction Detection in Requirements Engineering”. In: Proceedings of the Design Society 4 (Design 2024), pp. 2049–2058. URL: <https://doi.org/10.1017/pds.2024.207> (visited on 08/15/2025).

Gent, Edd (Jan. 28, 2025). Initiative Aims to Enable Ethical Coding LLMs: Software Heritage Wants to Put 22 billion Open-Source Files to Good Use. IEEE Spectrum. URL: <https://spectrum.ieee.org/coding-ai> (visited on 08/15/2025).

Green, Collin et al. (2024). “Developer Productivity for Humans, Part 7: Software Quality”. In: IEEE Software 41.1, pp. 25–30. URL: <https://doi.org/10.1109/MS.2023.3324830> (visited on 08/15/2025).

Greenstein, Shane (2024). “Navigating Applications Development in Generative AI”. In: IEEE Micro 44.4, pp. 122–124. URL: <https://doi.org/10.1109/MM.2024.3425209> (visited on 08/15/2025).

Greyling, Cobus (June 11, 2025a). AI Agents & Agentic AI: Understanding the Next Frontier in Artificial Intelligence. Medium. URL: <https://medium.com/p/bfe4b3c586a4> (visited on 08/15/2025).

— **(Feb. 12, 2025b).** From Handcrafted Workflows to AI Agents to Agentic Workflows. Medium. URL: <https://medium.com/p/49e8f85daf7d> (visited on 08/15/2025).

Gu, Xiaodong et al. (2025). “On the Effectiveness of Large Language Models in Domain-Specific Code Generation”. In: ACM Transactions on Software Engineering and Methodology 34.3, 78:1–78:22. URL: <https://doi.org/10.1145/3697012> (visited on 08/15/2025).

Guo, Daya et al. (2024). DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. Computing Research Repository (CoRR). URL: <https://doi.org/10.48550/arXiv.2401.14196> (visited on 08/15/2025).

Harding, William (Feb. 2025). AI Copilot Code Quality: Evaluating 2024’s Increased Defect Rate via Code Quality Metrics. Whitepaper. Seattle, WA: GitClear. URL: https://www.gitclear.com/ai_assistant_code_quality_2025_research (visited on 08/15/2025).

Harding, William and Kloster Matthew (Jan. 2024). Coding on Copilot: 2023 Data Shows Downward Pressure on Code Quality. Whitepaper. Seattle, WA: GitClear. URL: https://www.gitclear.com/coding_on_copilot_data_shows_ais_downward_pressure_on_code_quality (visited on 08/15/2025).

He, Junda, Christoph Treude, and David Lo (2025). “LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead”. In: ACM Transactions on Software Engineering and Methodology 34.5, 124:1–124:30. URL: <https://doi.org/10.1145/3712003> (visited on 08/15/2025).

Hey, Tobias et al. (2020). “NoRBERT: Transfer Learning for Requirements Classification”. In: IEEE 28th International Requirements Engineering Conference (RE ’20), pp. 169–179. DOI: 10.1109/RE48521.2020.00028. URL: <https://doi.org/10.5445/IR/1000150464> (visited on 08/15/2025).

Hong, Sirui et al. (2024). “MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework”. In: The Twelfth International Conference on Learning Representations (ICLR ’24). DOI: 10.48550/arXiv.2308.00352. URL: <https://openreview.net/pdf?id=VtmBAGCN7o> (visited on 08/15/2025).



Hou, Wenpin and Zhicheng Ji (2025). “Comparing Large Language Models and Human Programmers for Generating Programming Code”. In: *Advanced Science* 12.8, 202412279:1–202412279:12. URL: <https://doi.org/10.1002/advs.202412279> (visited on 08/15/2025).

Hou, Xinyi et al. (2024). “Large Language Models for Software Engineering: A Systematic Literature Review”. In: *ACM Transactions on Software Engineering and Methodology* 33.8, 220:1–220:79. URL: <https://doi.org/10.1145/3695988> (visited on 08/15/2025).

Huckins, Grace (June 12, 2025). Are We Ready to Hand AI Agents the Keys? MIT Technology Review. URL: <https://www.technologyreview.com/2025/06/12/1118189/> (visited on 08/15/ 2025).

Hurlburt, George F. (2025). “Why Bring Science to the Forefront?” In: *Computer* 58.5, pp. 65–70. URL: <https://doi.org/10.1109/MC.2025.3544117> (visited on 08/15/2025).

Hüttel, Hans (2025). “On Program Synthesis and Large Language Models”. In: *Communications of the ACM* 68.1, pp. 33–35. DOI: 10.1145/3680410. URL: <https://cacm.acm.org/opinion/on-program-synthesis-and-large-language-models/> (visited on 08/15/2025).

Hymel, Cory (2024). The AI-Native Software Development Lifecycle: A Theoretical and Practical New Methodology. Computing Research Repository (CoRR). URL: <https://doi.org/10.48550/arXiv.2408.03416> (visited on 08/15/2025).

Inman, Sarah, Sarah D’Angelo, and Bogdan Vasilescu (2024). “Developer Productivity for Humans, Part 8: Creativity in Software Engineering”. In: *IEEE Software* 41.2, pp. 11–16. URL: <https://doi.org/10.1109/MS.2023.3340831> (visited on 08/15/2025).

Jackson, Victoria, André van der Hoek, et al. (2025). “Introduction to the Special Issue on Creativity in Software Engineering”. In: *IEEE Software* 42.3, pp. 27–32. URL: <https://doi.org/10.1109/MS.2025.3540536> (visited on 08/15/2025).

Jackson, Victoria, Bogdan Vasilescu, et al. (2025). “The Impact of Generative AI on Creativity in Software Development: A Research Agenda”. In: *ACM Transactions on Software Engineering and Methodology* 34.5, 133:1–133:28. URL: <https://doi.org/10.1145/3708523> (visited on 08/15/2025).

Jaspan, Ciera and Collin Green (2025). “Measuring Productivity: All Models Are Wrong, But Some Are Useful”. In: *IEEE Software* 42.2, pp. 13–17. URL: <https://doi.org/10.1109/MS.2024.3511735> (visited on 08/15/2025).

JetBrains (2023). Artificial Intelligence: The State of Developer Ecosystem in 2023. JetBrains. URL: <https://www.jetbrains.com/lp/devecosystem-2023/> (visited on 08/15/2025).

Ji, Yichao (July 18, 2025). Context Engineering for AI Agents: Lessons from Building Manus. Medium. URL: <https://medium.com/p/71883f0a67f2> (visited on 08/15/2025).

Jiang, Juyong et al. (2025). “A Survey on Large Language Models for Code Generation”. In: *ACM Transactions on Software Engineering and Methodology* 34. DOI: 10.48550/arXiv.2406.00515. URL: <https://doi.org/10.1145/3747588> (visited on 08/15/2025).

Jimenez, Carlos E. et al. (2024). “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” In: *The Twelfth International Conference on Learning Representations (ICLR 2024)*. Vienna, Austria. URL: <https://openreview.net/pdf?id=VTF8yNQM66> (visited on 08/15/2025).

Just, René, Darioush Jalali, and Michael D. Ernst (2014). “Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs”. In: *Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA ’14)*, pp. 437–440. URL: <https://doi.org/10.1145/2610384.2628055> (visited on 08/15/2025).



Kalliamvakou, Eirini (Sept. 7, 2022). Research: Quantifying GitHub Copilot's Impact on Developer Productivity and Happiness. The GitHub Blog. URL: <https://github.blog/2022-09-07-research-quantifying-github-copilots-impact-on-developer-productivity-and-happiness/> (visited on 08/15/2025).

Kampik, Timotheus et al. (2025). "Large Process Models: A Vision for Business Process Management in the Age of Generative AI". In: KI – Künstliche Intelligenz 39.2. tex.ids: Flint.KampikEtAl25sn, pp. 81–95. URL: <https://doi.org/10.1007/s13218-024-00863-8> (visited on 08/15/2025).

Karpathy, Andrej (Jan. 24, 2023). The Hottest New Programming Language Is English. URL: <https://x.com/karpathy/status/1617979122625712128> (visited on 08/15/2025).

— **(Feb. 3, 2025a).** There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. (@karpathy) Post on X (formerly Twitter). URL: <https://x.com/karpathy/status/1886192184808149383> (visited on 08/15/ 2025).

— **(June 16, 2025b).** Software in the Era of AI: Software Is Changing (Again). Keynote at Y Combinator (YC) AI Startup School in San Francisco. URL: <https://youtu.be/LCEmiRjPEtQ> (visited on 08/15/2025).

Karunaratne, Damith (May 20, 2025). Coding With Gemini Just Got Easier. Google Blog. URL: <https://blog.google/technology/developers/gemini-code-assist-updates-google-io-2025/> (visited on 08/15/2025).

Kersten, Mik (2018). Project to Product: How to Survive and Thrive in the Age of Digital Disruption with the Flow Framework. Portland, OR: IT Revolution Press.

Kessel, Marcus and Colin Atkinson (2025a). "Morescent GAI for Software Engineering". In: ACM Transactions on Software Engineering and Methodology 34.5, 123:1–123:17. URL: <https://doi.org/10.1145/3709354> (visited on 08/15/2025).

— **(2025b).** "N-Version Assessment and Enhancement of Generative AI: Differential GAI". In: IEEE Software 42.2, pp. 76–83. DOI: 10.48550/arXiv.2409.14071. URL: <https://doi.org/10.1109/MS.2024.3469388> (visited on 08/15/2025).

Khan, Tahha (Apr. 29, 2025). Cost Analysis: Implementing Generative AI in Your Organization. AlphaBOLD. URL: <https://www.alphabold.com/cost-analysis-implementing-generative-ai-in-your-organization/> (visited on 08/15/2025).

Khandabattu, Haritha et al. (Feb. 2025). Critical Capabilities for AI Code Assistants. Report. Gartner Research. URL: <https://www.gartner.com/doc/5683419> (visited on 08/15/2025).

Khojah, Ranim et al. (2024). "Beyond Code Generation: An Observational Study of ChatGPT Usage in Software Engineering Practice". In: Proceedings of the ACM on Software Engineering 1 (FSE), pp. 1819–1840. URL: <https://doi.org/10.1145/3660788> (visited on 08/15/2025).

Kilpatrick, Logan (Apr. 29, 2024). Multimodal AI: How Multimodal AI Helps Business. Google Cloud. URL: <https://cloud.google.com/transform/how-multimodal-ai-helps-business> (visited on 08/15/2025).

Kim, Eugene (May 6, 2025). Amazon is Working on a Secret Project Called 'Kiro,' a New Tool that Uses AI Agents to Streamline Software Coding. Business Insider. URL: <https://www.businessinsider.com/amazon-kiro-project-ai-agents-software-coding-2025-5> (visited on 08/15/2025).

Kim, Gene and Steve Yegge (2025). Vibe Coding. Building Production-Grade Software with GenAI, Chat, Agents, and Beyond. Portland, OR: IT Revolution Press. URL: [https:// itrevolution.com/product/vibe-coding-book/](https://itrevolution.com/product/vibe-coding-book/) (visited on 08/15/2025).

Kokotajlo, Daniel et al. (Apr. 2025). AI 2027. Technical Report. AI Futures project. URL: <https://ai-2027.com/> (visited on 08/15/2025).



Koziolek, Anne et al. (Feb. 2025). Grand Challenges 2025: Fully Automated Software Development? German Informatics Society (GI – Gesellschaft für Informatik). URL: https://gi.de/fileadmin/GI/Projekte/Grand-Challenges/Fully_Automated_Software_Development_-_Longform_for_Download.pdf (visited on 08/15/2025).

Krauß, Veronika et al. (2025). ““Create a Fear of Missing Out” – ChatGPT Implements Unsolicited Deceptive Designs in Generated Websites Without Warning”. In: Proceedings of the 2025 ACM CHI Conference on Human Factors in Computing Systems, 857:1–857:20. URL: <https://doi.org/10.1145/3706598.3713083> (visited on 08/15/2025).

Kwa, Thomas et al. (2025). Measuring AI Ability to Complete Long Tasks. Computing Research Repository (CoRR). URL: <https://doi.org/10.48550/arXiv.2503.14499> (visited on 08/15/2025).

LangChain (June 23, 2025). The Rise of “Context Engineering”. LangChain Blog. URL: <https://blog.langchain.com/the-rise-of-context-engineering/> (visited on 08/15/2025).

Leblond, Rémi et al. (Dec. 2023). AlphaCode 2 Technical Report. Google DeepMind. URL: https://storage.googleapis.com/deepmind-media/AlphaCode2/AlphaCode2_Tech_Report.pdf (visited on 08/15/2025).

Lewis, Patrick et al. (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: Proceedings of the 34th International Conference on Neural Information Processing Systems (NeurIPS ’20), pp. 9459–9474. URL: https://proceedings.neurips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html (visited on 08/15/2025).

Li, Guohao et al. (2023). “CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society”. In: Proceedings of the 37th International Conference on Neural Information Processing Systems (NeurIPS ’23). Vol. 36. Advances in Neural Information Processing Systems, pp. 51991–52008. DOI: 10.48550/arXiv.2303.17760. URL: https://proceedings.neurips.cc/paper_files/paper/2023/hash/a3621ee907def47c1b952ade25c67698-Abstract-Conference.html (visited on 08/15/2025).

Li, Yujia et al. (Dec. 9, 2022). “Competition-Level Code Generation with AlphaCode”. In: Science 378.6624, pp. 1092–1097. DOI: 10.48550/arXiv.2203.07814. URL: <https://doi.org/10.1126/science.abq1158> (visited on 08/15/2025).

Lin, Chin-Yew (2004). “ROUGE: A Package for Automatic Evaluation of Summaries”. In: Proceedings of the Workshop on Text Summarization Branches Out (WAS 2004), pp. 74–81. URL: <https://aclanthology.org/W04-1013/> (visited on 08/15/2025).

Liu, Yue et al. (2024). “Refining ChatGPT-Generated Code: Characterizing and Mitigating Code Quality Issues”. In: ACM Transactions on Software Engineering and Methodology 33.5, 116:1–116:26. URL: <https://doi.org/10.1145/3643674> (visited on 08/15/2025).

Lovely, Garrison (Mar. 19, 2025). AI Could Soon Tackle Projects That Take Humans Weeks. Nature. URL: <https://doi.org/10.1038/d41586-025-00831-8> (visited on 08/15/2025).

Lozhkov, Anton et al. (2024). StarCoder 2 and The Stack v2: The Next Generation. Computing Research Repository (CoRR). URL: <https://doi.org/10.48550/arXiv.2402.19173> (visited on 08/15/2025).

Lu, Yao et al. (2024). Computing in the Era of Large Generative Models: From Cloud-Native to AI-Native. Computing Research Repository (CoRR). URL: <https://doi.org/10.48550/arXiv.2401.12230> (visited on 08/15/2025).

Lyu, Michael R. et al. (2025). “Automatic Programming: Large Language Models and Beyond”. In: ACM Transactions on Software Engineering and Methodology 34.5, 140:1–140:33. URL: <https://doi.org/10.1145/3708519> (visited on 08/15/2025).



Ma, Qianou Christina, Tongshuang Wu, and Kenneth R. Koedinger (2023). Is AI the Better Programming Partner? Human-Human Pair Programming vs. Human-AI pAIr Programming. Computing Research Repository (CoRR). URL: <https://doi.org/10.48550/arXiv.2306.05153> (visited on 08/15/2025).

Masood, Adnan (May 9, 2025). Security Analysis and Validation of Generative-AI-Produced Code. Medium. URL: <https://medium.com/p/d4218078bd63> (visited on 08/15/2025).

Mastropaolo, Antonio (2025). “How Artificial Intelligence Is Reshaping Our Lives: A Framework to Unravel Its Complexities”. In: Computer 58.6, pp. 4–5. URL: <https://doi.org/10.1109/MC.2025.3549482> (visited on 08/15/2025).

Mastropaolo, Antonio, Camilo Escobar-Velásquez, and Mario Linares-Vásquez (2025). “From Triumph to Uncertainty: The Journey of Software Engineering in the AI Era”. In: ACM Transactions on Software Engineering and Methodology 34.5, 131:1–131:34. URL: <https://doi.org/10.1145/3709360> (visited on 08/15/2025).

Meyer, David (Apr. 4, 2024). The Cost of Training AI Could Soon Become Too Much to Bear. Fortune Media. URL: <https://fortune.com/2024/04/04/ai-training-costs-how-much-is-too-much-openai-gpt-anthropic-microsoft/> (visited on 08/15/2025).

Mitchell, Margaret et al. (2025). Fully Autonomous AI Agents Should Not be Developed. Computing Research Repository (CoRR). URL: <https://doi.org/10.48550/arXiv.2502.02649> (visited on 08/15/2025).

Natella, Roberto et al. (2024). “AI Code Generators for Security: Friend or Foe?” In: IEEE Security and Privacy 22.5, pp. 73–81. URL: <https://doi.org/10.1109/MSEC.2024.3355713> (visited on 08/15/2025).

Nguyen, Nhan and Sarah Nadi (2022). “An Empirical Evaluation of GitHub Copilot’s Code Suggestions”. In: Proceedings of the 19th International Conference on Mining Software Repositories (MSR ’22), pp. 1–5. URL: <https://doi.org/10.1145/3524842.3528470> (visited on 08/15/2025).

Nielsen, Jakob (July 16, 2023). AI Improves Employee Productivity by 66%. Nielsen Norman Group. URL: <https://www.nngroup.com/articles/ai-tools-productivity-gains/> (visited on 08/15/2025).

Nikolov, Stoyan et al. (2025). How Is Google Using AI for Internal Code Migrations? Computing Research Repository (CoRR). URL: <https://doi.org/10.48550/arXiv.2501.06972> (visited on 08/15/2025).

Noda, Abi et al. (2023). “DevEx – What Actually Drives Productivity: The Developer-Centric Approach to Measuring and Improving Productivity”. In: Queue 21.2, pp. 35–53. URL: <https://doi.org/10.1145/3595878> (visited on 08/15/2025).

Novikov, Alexander et al. (May 2025). AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. Whitepaper. Google DeepMind. URL: <https://deepmind.google/discover/blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/> (visited on 08/15/2025).

OpenAI et al. (2025). Competitive Programming with Large Reasoning Models. Computing Research Repository (CoRR). URL: <https://doi.org/10.48550/arXiv.2502.06807> (visited on 08/15/2025).

OpenAI (Aug. 7, 2025). GPT-5 System Card. OpenAI. URL: <https://openai.com/index/gpt-5-system-card/> (visited on 08/15/2025).

Ozkaya, Ipek (2023). “Can Architecture Knowledge Guide Software Development with Generative AI?” In: IEEE Software 40.5, pp. 4–8. URL: <https://doi.org/10.1109/MS.2023.3306641> (visited on 08/15/2025).

Panyam, Sriram and Praveen Gujar (2025). “How AI Agents Are Transforming Software Engineering and the Future of Product Development”. In: Computer 58.5, pp. 71–77. URL: <https://doi.org/10.1109/MC.2024.3488378> (visited on 08/15/2025).



Papineni, Kishore et al. (2002). “BLEU: A Method for Automatic Evaluation of Machine Translation”. In: Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL '02), pp. 311–318. URL: <https://doi.org/10.3115/1073083.1073135> (visited on 08/15/ 2025).

Pasquale, Liliana et al. (2025). “Challenges to Using Large Language Models in Code Generation and Repair”. In: IEEE Security & Privacy 23.2, pp. 81–88. URL: <https://doi.org/10.1109/MSEC.2025.3530488> (visited on 08/15/2025).

Patel, Harsh et al. (2025). “A State-of-the-Practice Release-Readiness Checklist for Generative AI-Based Software Products: A Gray Literature Survey”. In: IEEE Software 42.1, pp. 74–83. URL: <https://doi.org/10.1109/MS.2024.3440190> (visited on 08/15/2025).

Pearce, Hammond et al. (2025). “Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions”. In: Communications of the ACM 68.2, pp. 96–105. DOI: 10.1145/3610721. URL: <https://cacm.acm.org/research-highlights/asleep-at-the-keyboard-assessing-the-security-of-github-copilots-code-contributions/> (visited on 08/15/ 2025).

Pearson, Ben Lloyd (Apr. 25, 2025). Microsoft’s Strategy For Driving AI Adoption in Engineering Teams. LinearB. URL: <https://linearb.io/blog/microsoft-strategy-for-driving-ai-adoption> (visited on 08/15/2025).

Petridis, Savvas, Michael Terry, and Carrie J Cai (2024). “PromptInfuser: How Tightly Coupling AI and UI Design Impacts Designers’ Workflows”. In: Proceedings of the 2024 ACM Designing Interactive Systems Conference (DIS '24), pp. 743–756. URL: <https://doi.org/10.1145/3643834.3661613> (visited on 08/15/2025).

Pinto, Gustavo et al. (2024). “Lessons from Building StackSpot AI: A Contextualized AI Coding Assistant”. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '24), pp. 408–417. DOI: 10.48550/arXiv.2311.18450. URL: <https://doi.org/10.1145/3639477.3639751> (visited on 08/15/2025).

Porter, Dylan, Joanna F. DeFranco, and Phil Laplante (2025). “Requirements Specification Automated Quality Analysis: Past, Present, and Future”. In: Computer 58.1, pp. 101–104. URL: <https://doi.org/10.1109/MC.2024.3480629> (visited on 08/15/2025).

PwC (2024). 10 Ways GenAI Improves Software Development. PricewaterhouseCoopers. URL: <https://www.pwc.com/us/en/tech-effect/ai-analytics/generative-ai-for-software-development.html> (visited on 08/15/2025).

Qiu, Ketai et al. (2025). “From Today’s Code to Tomorrow’s Symphony: The AI Transformation of Developer’s Routine by 2030”. In: ACM Transactions on Software Engineering and Methodology 34.5, 121:1– 121:17. URL: <https://doi.org/10.1145/3709353> (visited on 08/15/2025).

Rao, Nikitha et al. (2023). “CAT-LM Training Language Models on Aligned Code and Tests”. In: Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE '23), pp. 409–420. URL: <https://doi.org/10.1109/ASE56229.2023.00193> (visited on 08/15/2025).

Rasmussen, Kris (June 26, 2024). Meet Figma AI: Empowering Designers with Intelligent Tools. Figma Blog. URL: <https://www.figma.com/blog/introducing-figma-ai/> (visited on 08/15/ 2025).

Relevantz (Nov. 19, 2024). How to Increase Developer Productivity with GenAI – We’ve Done It. Relevantz Blog. URL: <https://relevantz.com/blog/how-to-increase-developer-productivity-with-genai/> (visited on 08/15/2025).

Ren, Shuo et al. (2020). CodeBLEU: A Method for Automatic Evaluation of Code Synthesis. Computing Research Repository (CoRR). URL: <https://doi.org/10.48550/arXiv.2009.10297> (visited on 08/15/2025).



Robinson, Diana et al. (2025). “Requirements Are All You Need: The Final Frontier for End-User Software Engineering”. In: ACM Transactions on Software Engineering and Methodology 34.5, 144:1–144:22. URL: <https://doi.org/10.1145/3708524> (visited on 08/15/2025).

Romera-Paredes, Bernardino et al. (Jan. 18, 2024). “Mathematical Discoveries from Program Search with Large Language Models”. In: Nature 625.7995, pp. 468–475. URL: <https://doi.org/10.1038/s41586-023-06924-6> (visited on 08/15/2025).

Romero, Alberto (Aug. 9, 2025). GPT-5 Is Here: There’s Only One Feature Worth Writing About. Medium. URL: <https://medium.com/p/4aa4be1df15b> (visited on 08/15/2025).

Rossi, Francesca et al. (Mar. 2025). AAAI 2025 Presidential Panel on the Future of AI Research. Report. Association for the Advancement of Artificial Intelligence (AAAI). URL: <https://aaai.org/about-aaai/presidential-panel-on-the-future-of-ai-research/> (visited on 08/15/ 2025).

Russo, Daniel (2024). “Navigating the Complexity of Generative AI Adoption in Software Engineering”. In: ACM Transactions on Software Engineering and Methodology 33.5, 135:1–135:50. URL: <https://doi.org/10.1145/3652154> (visited on 08/15/2025).

Sajja, Ayyappa et al. (2024). “Integrating Generative AI into the Software Development Lifecycle: Impacts on Code Quality and Maintenance”. In: International Journal of Science and Research Archive 13.1, pp. 1952–1960. URL: <https://doi.org/10.30574/ijrsra.2024.13.1.1837> (visited on 08/15/ 2025).

Saleem, Summra, Muhammad Nabeel Asim, and Andreas Dengel (2025). “PassionNet: An Innovative Framework for Duplicate and Conflicting Requirements Identification”. In: Expert Systems with Applications 293, 128684:1–128684:19. URL: <https://doi.org/10.1016/j.eswa.2025.128684> (visited on 08/15/ 2025).

Saleem, Summra, Muhammad Nabeel Asim, Ludger van Elst, and Andreas Dengel (2023). “FNReq-Net: A Hybrid Computational Framework for Functional and Non-Functional Requirements Classification”. In: Journal of King Saud University – Computer and Information Sciences 35.8, 101665:1–101665:14. URL: <https://doi.org/10.1016/j.jksuci.2023.101665> (visited on 08/15/2025).

— **(2025).** “Generative Language Models Potential for Requirement Engineering Applications: Insights into Current Strengths and Limitations”. In: Complex & Intelligent Systems 11.6, 278:1–278:22. URL: <https://doi.org/10.1007/s40747-024-01707-6> (visited on 08/15/2025).

Saleem, Summra, Muhammad Nabeel Asim, Ludger van Elst, Markus Junker, et al. (2024). “MLR-predictor: A Versatile and Efficient Computational Framework for Multi-Label Requirements Classification”. In: Frontiers in Artificial Intelligence 7, 1481581:1–1481581:26. URL: <https://doi.org/10.3389/frai.2024.1481581> (visited on 08/15/2025).

Salesforce (May 23, 2024). What Is Agentforce Assistant? (Formerly Einstein Copilot). Salesforce.com. URL: <https://www.salesforce.com/agentforce/einstein-copilot/> (visited on 08/15/2025).

Sapkota, Ranjan, Konstantinos I. Roumeliotis, and Manoj Karkee (2025). AI Agents vs. Agentic AI: A Conceptual Taxonomy, Applications and Challenges. Computing Research Repository (CoRR). URL: <https://doi.org/10.48550/arXiv.2505.10468> (visited on 08/15/2025).

Sasaki, Yuya et al. (2025). “Landscape and Taxonomy of Prompt Engineering Patterns in Software Engineering”. In: IT Professional 27.1, pp. 41–49. URL: <https://doi.org/10.1109/MITP.2024.3525458> (visited on 08/15/2025).

Sauvola, Jaakko et al. (2024). “Future of Software Development with Generative AI”. In: Automated Software Engineering 31.1. URL: <https://doi.org/10.1007/s10515-024-00426-z> (visited on 08/15/2025).



Shen, Minjie et al. (2025). From Mind to Machine: The Rise of Manus AI as a Fully Autonomous Digital Agent. Computing Research Repository (CoRR). URL: <https://doi.org/10.48550/arXiv.2505.02024> (visited on 08/15/2025).

Shi, Jieke, Zhou Yang, and David Lo (2025). “Efficient and Green Large Language Models for Software Engineering: Literature Review, Vision, and the Road Ahead”. In: ACM Transactions on Software Engineering and Methodology 34.5, 137:1–137:22. URL: <https://doi.org/10.1145/3708525> (visited on 08/15/2025).

Shoham, Yoav (July 3, 2025). Don’t Let Hype About AI Agents Get Ahead of Reality. MIT Technology Review. URL: <https://www.technologyreview.com/2025/07/03/1119545/> (visited on 08/15/2025).

Singla, Alex et al. (Mar. 2025). The State of AI: How Organizations Are Rewiring To Capture Value. Survey Report. McKinsey & Company. URL: <https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai> (visited on 08/15/2025).

Sivasubramanian, Swami (Apr. 13, 2023). *Announcing New Tools for Building with Generative AI on AWS.* AWS Artificial Intelligence Blog. URL: <https://aws.amazon.com/blogs/machine-learning/announcing-new-tools-for-building-with-generative-ai-on-aws/> (visited on 08/15/2025).

Smith, Matthew S. (June 21, 2025). Best AI Coding Tools: Claude Code, Windsurf, and VSCode. IEEE Spectrum. URL: <https://spectrum.ieee.org/best-ai-coding-tools> (visited on 08/15/2025).

Song, Da et al. (2024). “LUNA: A Model-Based Universal Analysis Framework for Large Language Models”. In: IEEE Transactions on Software Engineering 50.7, pp. 1921–1948. URL: <https://doi.org/10.1109/TSE.2024.3411928> (visited on 08/15/2025).

Spinellis, Diomidis (2024). “Pair Programming with Generative AI”. In: IEEE Software 41.3, pp. 16–18. URL: <https://doi.org/10.1109/MS.2024.3363848> (visited on 08/15/2025).

Sponheim, Caleb and Megan Brown (Apr. 12, 2024). AI UX-Design Tools Are Not Ready for Primetime: Status Update. Nielsen Norman Group. URL: <https://www.nngroup.com/articles/ai-design-tools-not-ready/> (visited on 08/15/2025).

Stack Overflow (May 29, 2024). Developers Get by with a Little Help From AI: Stack Overflow Knows Code Assistant Pulse Survey Results. Stack Overflow Blog. URL: <https://stackoverflow.blog/2024/05/29/developers-get-by-with-a-little-help-from-ai-stack-overflow-knows-code-assistant-pulse-survey-results/> (visited on 08/15/2025).

Sun, Yeali S. et al. (2024). “Unleashing Malware Analysis and Understanding with Generative AI”. In: IEEE Security & Privacy 22.3, pp. 12–23. URL: <https://doi.org/10.1109/MSEC.2024.3384415> (visited on 08/15/2025).

Surapaneni, Rao, Miku Jha, et al. (Apr. 25, 2025). Announcing the Agent2Agent Protocol (A2A). Google for Developers Blog. URL: <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/> (visited on 08/15/2025).

Surapaneni, Rao and Philip Stephens (Aug. 1, 2025). Announcing a Complete Developer Toolkit for Scaling A2A Agents on Google Cloud. Google Cloud Blog. URL: <https://cloud.google.com/blog/products/ai-machine-learning/agent2agent-protocol-is-getting-an-upgrade> (visited on 08/15/2025).

Swaminathan, Nikhil and Deepak Singh (July 14, 2025). Introducing Kiro: A New Agentic IDE That Works Alongside You from Prototype to Production. AWS Kiro Blog. URL: <https://kiro.dev/blog/introducing-kiro/> (visited on 08/15/2025).

Tan, Kwan Wei Kevin (Mar. 14, 2025). Anthropic’s CEO says that in 3 to 6 months, AI will be writing 90%



of the code software developers were in charge of. Business Insider. URL: <https://www.businessinsider.com/anthropic-ceo-ai-90-percent-code-3-to-6-months-2025-3> (visited on 08/15/2025).

Tantithamthavorn, Chakkrit, Jürgen Cito, et al. (2023). “Explainable AI for SE: Challenges and Future Directions”. In: IEEE Software 40.3, pp. 29–33. URL: <https://doi.org/10.1109/MS.2023.3246686> (visited on 08/15/2025).

Tantithamthavorn, Chakkrit, Jirayus Jiarapakdee, and John Grundy (2021). “Actionable Analytics: Stop Telling Me What It Is; Please Tell Me What to Do”. In: IEEE Software 38.4, pp. 115–120. URL: <https://doi.org/10.1109/MS.2021.3072088> (visited on 08/15/2025).

Tantithamthavorn, Chakkrit Kla et al. (2025). “MLOps, LLMOps, FMOps, and Beyond”. In: IEEE Software 42.1, pp. 26–32. URL: <https://doi.org/10.1109/MS.2024.3477014> (visited on 08/15/2025).

Team Ascend (Apr. 7, 2025). Preparing Your Team for AI Adoption: Microsoft 365 Copilot. Team Ascend Blog. URL: <https://blog.teamascend.com/preparing-your-team-for-ai-adoption> (visited on 08/15/2025).

Tenbergen, Bastian and Stephan Krusche (2024). “The Future of Software Engineering Education and Training in the Age of AI”. In: IEEE Software 41.2, pp. 28–29. URL: <https://doi.org/10.1109/MS.2023.3345960> (visited on 08/15/2025).

Treude, Christoph and Marco A. Gerosa (2025). “How Developers Interact with AI: A Taxonomy of Human- AI Collaboration in Software Engineering”. In: Proceedings of the 2nd ACM international Conference on AI Foundation Models and Software Engineering (FORGE ’25). URL: <https://doi.org/10.48550/arXiv.2501.08774> (visited on 08/15/2025).

Tricentis (2021). AI-Powered Test Automation for the Next Generation of Applications. Tricentis.com. URL: <https://www.tricentis.com/resources/ai-powered-test-automation/> (visited on 08/15/2025).

Trueman, Charlotte (Apr. 19, 2023). Atlassian Adds Generative AI to Its Cloud Workforce Management Offerings. Computerworld. URL: <https://www.computerworld.com/article/1623393/> (visited on 08/15/2025).

Utle, Jeremy D. (May 5, 2025a). It’s Not an AI Problem. It’s A You Problem. Jeremy Utley Design. URL: <https://www.jeremyutley.design/blog/teammate-not-technology> (visited on 08/15/2025).

— **(Apr. 28, 2025b).** Teammate, Not Technology. Jeremy Utley Design. URL: <https://www.jeremyutley.design/blog/teammate-not-technology> (visited on 08/15/2025).

Vaidya, Jaideep and Hafiz Asif (2023). “A Critical Look at AI-Generated Software: Coding with the New AI Tools is Both Irresistible and Dangerous”. In: IEEE Spectrum 60.7, pp. 34–39. DOI: 10.1109/MSPEC.2023.10177044. URL: <https://spectrum.ieee.org/ai-software> (visited on 08/15/2025).

Valliani, Jamil (May 20, 2025a). Atlassian Recognized as an Emerging Leader in the 2025 Gartner Innovation Guide for Generative AI Technologies. Work Life by Atlassian. URL: <https://www.atlassian.com/blog/announcements/gartner-genai-emerging-leader> (visited on 08/15/2025).

— **(Apr. 9, 2025b).** Bringing the Magic of Human-AI Collaboration to Every Team: Powerful Rovo Search, Chat, and Agents for All. Work Life by Atlassian. URL: <https://www.atlassian.com/blog/announcements/team25-rovo-for-all> (visited on 08/15/2025).

Vaswani, Ashish et al. (2017). “Attention is All you Need”. In: Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS ’17), pp. 6000–6010. URL: https://papers.nips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html (visited on 08/15/2025).

Wahlster, Wolfgang (July 5, 2023). “Advanced Language Models: From Super-Parrots to Human-Like



Dialog Understanding”. The Hessian AICon 2023 (Darmstadt, Germany), Invited talk. URL: <https://hessian.ai/de/aicon-2023/tag-2/https://hessian.ai/de/aicon-2023/tag-2/> (visited on 08/15/2025).

Waldo, Jim and Soline Boussard (2025). “GPTs and Hallucination”. In: Communications of the ACM 68.1, pp. 40–45. DOI: 10.1145/3703757. URL: <https://cacm.acm.org/practice/gpts-and-hallucination/> (visited on 08/15/2025).

Washizaki, Hironori, ed. (2024). Guide to the Software Engineering Body of Knowledge (SWEBOK Guide). 4.0. Los Alamitos, CA: IEEE Computer Society. URL: <http://swebok.org/> (visited on 08/15/2025).

Wei, Bingyang (2024). “Requirements Are All You Need: From Requirements to Code with LLMs”. In: IEEE 32nd International Requirements Engineering Conference (RE ’24), pp. 416–422. DOI: <https://doi.org/10.48550/arXiv.2406.10101> URL: <https://doi.org/10.1109/RE59067.2024.00049> (visited on 08/15/2025).

Wei, Jason et al. (2022). “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: Proceedings of the 36th International Conference on Neural Information Processing Systems, pp. 24824–24837. URL: https://proceedings.neurips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html (visited on 08/15/2025).

Wei, Jialiang et al. (2025). “On AI-Inspired User Interface Design”. In: IEEE Software 42.3, pp. 50–58. URL: <https://doi.org/10.1109/MS.2025.3536838> (visited on 08/15/2025).

Weingartner, Michael and Richard Seroter (May 11, 2023). Introducing Duet AI for Google Cloud – an AI-Powered Collaborator. Google Cloud Blog. URL: <https://cloud.google.com/blog/products/application-modernization/introducing-duet-ai-for-google-cloud> (visited on 08/15/2025).

Welsh, Matt (2023). “The End of Programming”. In: Communications of the ACM 66.1, pp. 34–35. DOI: 10.1145/3570220. URL: <https://cacm.acm.org/opinion/the-end-of-programming/> (visited on 08/15/2025).

Wolf, Yotam et al. (2024). “Fundamental Limitations of Alignment in Large Language Models”. In: Proceedings of the 41st International Conference on Machine Learning (ICML’24). Proceedings of Machine Learning Research, pp. 53079–53112. URL: <https://proceedings.mlr.press/v235/wolf24a.html> (visited on 08/15/2025).

Wu, Scott (Mar. 12, 2024). Introducing Devin, the First AI Software Engineer. Cognition.AI Blog. URL: <https://cognition.ai/blog/introducing-devin/> (visited on 08/15/2025).

— (July 14, 2025). Cognition’s Acquisition of Windsurf. Cognition.AI Blog. URL: <https://cognition.ai/blog/windsurf/> (visited on 08/15/2025).

Xu, Frank F., Bogdan Vasilescu, and Graham Neubig (2022). “In-IDE Code Generation from Natural Language: Promise and Challenges”. In: ACM Transactions on Software Engineering and Methodology 31.2, 29:1–29:47. URL: <https://doi.org/10.1145/3487569> (visited on 08/15/2025).

Yang, Yanming et al. (2022). “A Survey on Deep Learning for Software Engineering”. In: ACM Computing Surveys 54.10, 206:1–206:73. URL: <https://doi.org/10.1145/3505243> (visited on 08/15/2025).

Yu, Shengcheng et al. (2023). “LLM for Test Script Generation and Migration: Challenges, Capabilities, and Opportunities”. In: Proceedings of the IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS 2023), pp. 206–217. URL: <https://doi.org/10.1109/QRS60937.2023.00029> (visited on 08/15/2025).

Yu, Yongda et al. (2025). “Fine-Tuning Large Language Models to Improve Accuracy and Comprehensibility of Automated Code Review”. In: ACM Transactions on Software Engineering and Methodology 34.1, 14:1–14:26. URL: <https://doi.org/10.1145/3695993> (visited on 08/15/2025).



Yuan, Xingdi et al. (Mar. 2025). debug-gym: A Text-Based Environment for Interactive Debugging. URL: <https://doi.org/10.48550/arXiv.2503.21557> (visited on 08/15/2025).

Zhang, Yue et al. (2024). “Does Generative AI Generate Smells Related to Container Orchestration?: An Exploratory Study with Kubernetes Manifests”. In: Proceedings of the 21st International Conference on Mining Software Repositories (MSR '24), pp. 192–196. URL: <https://doi.org/10.1145/3643991.3645079> (visited on 08/15/2025).

Zhang, Ziyin et al. (Sept. 2024). “Unifying the Perspectives of NLP and Software Engineering: A Survey on Language Models for Code”. In: Transactions on Machine Learning Research. URL: <https://openreview.net/pdf?id=hkNnGqZnpa> (visited on 08/15/2025).

Zheng, Zihan et al. (2025). LiveCodeBench Pro: How Do Olympiad Medalists Judge LLMs in Competitive Programming? Computing Research Repository (CoRR). URL: <https://doi.org/10.48550/arXiv.2506.11928> (visited on 08/15/2025).

Zhou, Bingzhe et al. (2023). “Hybrid API Migration: A Marriage of Small API Mapping Models and Large Language Models”. In: Proceedings of the 14th Asia-Pacific Symposium on Internetware, pp. 12–21. URL: <https://doi.org/10.1145/3609437.3609466> (visited on 08/15/2025).

Zhu, Qihao et al. (2021). “A Syntax-Guided Edit Decoder for Neural Program Repair”. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (SEC/FSE '21), pp. 341–353. URL: <https://doi.org/10.1145/3468264.3468544> (visited on 08/15/2025).

Ziegler, Albert et al. (2022). “Productivity Assessment of Neural Code Completion”. In: Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming (MAPS '22), pp. 21–29. URL: <https://doi.org/10.1145/3520312.3534864> (visited on 08/15/2025).

Zorpette, Glenn (July 2025). “The Data – The Future of AI: LLMs at Warp Speed – By 2030, LLMs May Tackle Monthlong Tasks in Hours”. In: IEEE Spectrum 62.7, pp. 14–15. DOI: 10.1109/MSPEC.2025.11074452 URL: <https://spectrum.ieee.org/large-language-model-performance> (visited on 08/15/2025).





8. Authors



Dr. Alassane Ndiaye

Senior Software Engineer (R&D), DFKI
alassane.ndiaye@dfki.de



Xhorxhina Taraj

Microsoft Cloud Advisor – Data & AI, Accenture
xhorxhina.taraj@accenture.com



Bostjan Robeznik

AI Enterprise Solution Architect, Accenture
bostjan.robeznik@accenture.com



Gerd Herzog

Chief Software Engineer, DFKI
gerd.hertzog@dfki.de



Dr. Christian Gerber

Managing Director AI Solutioning, Accenture
christian.gerber@accenture.com



Dr. Hendrik Purwins

Senior AI Manager, Accenture
hendrik.purwins@accenture.com